```
package nl.saxion.gameapp;

import com.badlogic.gdx.*;
import com.badlogic.gdx.audio.Music;
import com.badlogic.gdx.audio.Sound;
import com.badlogic.gdx.backends.lwjgl3.Lwjgl3Application;
import com.badlogic.gdx.backends.lwjgl3.Lwjgl3ApplicationConfiguration;
import com.badlogic.gdx.backends.lwjgl3.Lwjgl3Graphics;
import com.badlogic.gdx.files.FileHandle;
import com.badlogic.gdx.graphics.*;
import com.badlogic.gdx.graphics.g2d.*;
import com.badlogic.gdx.graphics.g2d.freetype.FreeTypeFontGenerator;
import com.badlogic.gdx.graphics.glutils.ShapeRenderer;
import com.badlogic.gdx.math.*;
import com.badlogic.gdx.scenes.scene2d.Actor;
import com.badlogic.gdx.scenes.scene2d.Stage;
import com.badlogic.gdx.scenes.scene2d.ui.*;
import com.badlogic.gdx.scenes.scene2d.utils.ChangeListener;
import com.badlogic.gdx.utils.Array;
import com.badlogic.gdx.utils.Json;
import com.badlogic.gdx.utils.ShortArray;
import com.esotericsoftware.kryo.Kryo;
import com.esotericsoftware.kryo.io.Input;
import com.esotericsoftware.kryo.io.Output;
import nl.saxion.gameapp.animation.Interpolator;
import nl.saxion.gameapp.animation.SpriteAnimation;
import nl.saxion.gameapp.screens.extensions.HUD;
import nl.saxion.gameapp.screens.extensions.WorldSize;
import nl.saxion.gameapp.utils.TailwindColors;
import nl.saxion.gameapp.utils.Timer;

import java.io.File;
import java.io.IOException;
import java.io.InputStream;
import java.nio.file.StandardCopyOption;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
```

```
 * A static utility wrapper around the LibGDX framework that simplifies game
development
 * and screen management.
 *
 * <p>
 * Typical usage involves calling {@link #start(String, int, int, int, boolean, String)} to
 * initialize the game and then using {@link #switchScreen(String)} to transition between
 * different game screens.
 * </p>
 *
 * <p><strong>Example:</strong></p>
 * <pre>{@code
 * public class MyGame {
 *     public static void main(String[] args) {
 *         GameApp.start("My Game", 1280, 720, 60, false, "MainMenu");
 *     }
 * }
 * }</pre>
 *
 * <p>
 * Screens must be registered before switching, typically through a static registration
 * method or within the game's initialization logic.
 * </p>
 *
 * @author Evert Duipmans (e.f.duipmans@saxion.nl)
 */
public class GameApp {
    private static GameImpl gameInstance;
    private static final Map<String, Screen> screens = new HashMap<>();
    private static final Map<String, Color> namedColors = new HashMap<>();
    private static final Map<String, Texture> textures = new HashMap<>();
    private static final Map<String, Sound> sounds = new HashMap<>();
    private static final Map<String, Music> music = new HashMap<>();
    private static final Map<String, BitmapFont> fonts = new HashMap<>();
    private static final Map<String, Object> store = new HashMap<>();
    private static final Map<String, Interpolator> interpolators = new HashMap<>();
    private static final Map<String, TextureRegion[][]> spritesheets = new HashMap<>();
    private static final Map<String, SpriteAnimation> animations = new HashMap<>();
    private static final Map<String, TextureAtlas> atlases = new HashMap<>();
    private static final Map<String, ParticleEffectPool> particleEffectPools = new
HashMap<>();
```

```
  private static final Map<Long, ParticleEffectPool.PooledEffect> activeParticleEffects =
new HashMap<>();
  private static final Map<String, Timer> timers = new HashMap<>();
  private static final Map<String, Skin> skins = new HashMap<>();
  private static final Map<String, Actor> uiElements = new HashMap<>();
  private static long nextParticleId = 1;

  /**
   * Necessary for rendering shapes in an efficient way.
   */
  private static ShapeRenderer shapeRenderer;

  /**
   * Necessary for rendering sprites in an efficient way.
   */
  private static SpriteBatch spriteBatch;

  /**
   * Keeps track if the masking mode is active.
   */
  private static boolean maskActive = false;

  /**
   * Inner class that extends the Game object of LibGDX
   * Used for initializing the game and loading the first screen.
   * This class may be needed when you want to deploy to other platforms than desktop!
   */
  public static class GameImpl extends Game {
    private final String firstScreen;

    public GameImpl(String firstScreen) {
      this.firstScreen = firstScreen;
    }

    @Override
    public void create() {
      shapeRenderer = new ShapeRenderer();
      spriteBatch = new SpriteBatch();

      // Show the first registered screen
      if (!screens.isEmpty() && screens.containsKey(firstScreen)) {
```

```
            switchScreen(firstScreen);
        } else {
            throw new IllegalStateException(String.format("SaxionGameApp cannot be
started, because the screen \"%s\" not found.", firstScreen));
        }

        // Add the default font
        fonts.put("default", new BitmapFont());
    }

    @Override
    public void dispose() {
        if (shapeRenderer != null) shapeRenderer.dispose();
        if (spriteBatch != null) spriteBatch.dispose();

        // Clear sounds and music
        for (Sound sound : sounds.values()) {
            sound.dispose();
        }
        for (Music music : music.values()) {
            music.dispose();
        }
        sounds.clear();
        music.clear();

        // Clear spritesheets
        for (String spritesheet : spritesheets.keySet()) {
            disposeSpritesheet(spritesheet);
        }

        // Clear atlases
        for (TextureAtlas atlas : atlases.values()) {
            atlas.dispose();
        }

        // Clear textures
        for (Texture t : textures.values()) {
            t.dispose();
        }
        textures.clear();
```

```
        // Clear fonts
        for (BitmapFont font : fonts.values()) {
          font.dispose();
        }
        fonts.clear();

        // Clear animations
        animations.clear();

        // Clear store
        store.clear();

        // Clear particle effects
        particleEffectPools.clear();
        activeParticleEffects.clear();
      }
    }

    /**
     * Configure the game application and start it immediately.
     * <p>
     * Developer info: This method creates and launches a new {@link Lwjgl3Application} instance using the provided
     * title, resolution, frame rate, and display mode settings. The game begins by displaying
     * the screen identified by {@code firstScreenName}.
     * </p>
     *
     * @param title         the title of the game window
     * @param width         the width of the game window in pixels
     * @param height        the height of the game window in pixels
     * @param fps           the target frames per second (FPS) for rendering
     * @param fullScreen    {@code true} to start the game in fullscreen mode; {@code false} for windowed mode
     * @param firstScreenName the name of the first screen to display when the game starts
     * @throws IllegalStateException if the game has already been started
     */
    public static void start(String title, int width, int height, int fps, boolean fullScreen, String firstScreenName) {
      if (gameInstance != null) {
```

```
            throw new IllegalStateException("GameApp already started.");
        }

        Lwjgl3ApplicationConfiguration config = new Lwjgl3ApplicationConfiguration();
        config.setTitle(title);
        config.setWindowedMode(width, height);
        config.useVsync(true);
        config.setForegroundFPS(fps);

        config.setBackBufferConfig(
            8, 8, 8, 8,  // 8 bits per channel RGBA
            16,       // depth buffer
            8,        // stencil buffer
            4         // MSAA 4x
        );

        if (fullScreen) {
            config.setFullscreenMode(Lwjgl3ApplicationConfiguration.getDisplayMode());
        }

        // Register default colors
        TailwindColors.registerDefaultColors();

        // Start application
        gameInstance = new GameImpl(firstScreenName);
        new Lwjgl3Application(gameInstance, config);
    }

    /**
     * Creates a new GameImpl instance for cross-platform backends (Android, iOS, WebGL).
     * Screens must be registered separately before starting the game.
     *
     * @param firstScreenName the name of the first screen to display
     * @return the created GameImpl instance
     */
    public static GameImpl createCrossPlatformGame(String firstScreenName) {
        // Register default colors
        TailwindColors.registerDefaultColors();

        gameInstance = new GameImpl(firstScreenName);
```

```
    return gameInstance;
  }

  /**
   * Exits the game application.
   * <p>
   * This method signals LibGDX to terminate the application and close the game window.
   * Any cleanup logic in {@link com.badlogic.gdx.ApplicationListener#dispose()} will be executed.
   * </p>
   */
  public static void quit() {
    Gdx.app.exit();
  }

  /**
   * Switches the game window to fullscreen mode using the current display mode.
   * <p>
   * This method only works on desktop platforms using the Lwjgl3 backend.
   * </p>
   */
  public static void switchToFullscreen() {
    if (Gdx.graphics instanceof Lwjgl3Graphics graphics) {
      graphics.setFullscreenMode(Lwjgl3ApplicationConfiguration.getDisplayMode());
    } else {
      throw new RuntimeException("Current game backend does not support display mode fullscreen.");
    }
  }

  /**
   * Switches the game window to windowed mode with the specified width and height.
   * <p>
   * This method only works on desktop platforms using the Lwjgl3 backend.
   * </p>
   *
   * @param width  the width of the window in pixels
   * @param height the height of the window in pixels
   * @throws RuntimeException if the current backend does not support windowed mode changes
```

```
     */
    public static void switchToWindowedMode(int width, int height) {
        if (Gdx.graphics instanceof Lwjgl3Graphics graphics) {
            graphics.setWindowedMode(width, height);
        } else {
            throw new RuntimeException("Current game backend does not support windowed
mode.");
        }
    }

    /**
     * Sets whether the game window can be resized by the user.
     * <p>
     * This method only works on desktop platforms using the Lwjgl3 backend.
     * </p>
     *
     * @param resizable {@code true} to allow resizing, {@code false} to disable
     * @throws RuntimeException if the current backend does not support changing
resizable property
     */
    public static void setResizable(boolean resizable) {
        if (Gdx.graphics instanceof Lwjgl3Graphics graphics) {
            graphics.setResizable(resizable);
        } else {
            throw new RuntimeException("Current game backend does not support changing
resizable property.");
        }
    }

    /**
     * Changes the title of the game window.
     * <p>
     * This method only works on desktop platforms using the Lwjgl3 backend.
     * </p>
     *
     * @param title the new window title
     * @throws RuntimeException if the current backend does not support changing the
window title
     */
    public static void setTitle(String title) {
        if (Gdx.graphics instanceof Lwjgl3Graphics graphics) {
```

```
        graphics.getWindow().setTitle(title);
      } else {
        throw new RuntimeException("Current game backend does not support changing
the window title.");
      }
    }

    /**
     * Checks if the game is currently in fullscreen mode.
     * <p>
     * This method only works on desktop platforms using the Lwjgl3 backend.
     * </p>
     *
     * @return {@code true} if the game is fullscreen, {@code false} otherwise
     * @throws RuntimeException if the current backend does not support fullscreen
mode detection
     */
    public static boolean isFullscreen() {
      if (Gdx.graphics instanceof Lwjgl3Graphics graphics) {
        return graphics.isFullscreen();
      } else {
        throw new RuntimeException("Current game backend does not support fullscreen
detection.");
      }
    }

    /**
     * Sets the target frames per second (FPS) for rendering.
     * <p>
     * This method only works on desktop platforms. Changing the FPS dynamically can
help
     * reduce CPU/GPU usage or control the speed of the game loop.
     * </p>
     *
     * @param fps the target frames per second (must be > 0)
     * @throws IllegalArgumentException if {@code fps} is not positive
     */
    public static void setFPS(int fps) {
      if (fps <= 0) {
        throw new IllegalArgumentException("FPS must be positive.");
      }
```

```
    Gdx.graphics.setForegroundFPS(fps);
}

/**
 * Enables or disables vertical synchronization (VSync).
 * <p>
 * When VSync is enabled, the frame rate is synchronized to the monitor's refresh rate
 * to prevent screen tearing. This method only works on desktop platforms.
 * </p>
 *
 * @param vsync {@code true} to enable VSync, {@code false} to disable
 */
public static void setVSync(boolean vsync) {
    Gdx.graphics.setVSync(vsync);
}

/**
 * Register a screen with a unique name.
 * @param name Name of the screen
 * @param screen Screen object
 */
public static void addScreen(String name, Screen screen) {
    screens.put(name, screen);
}

/**
 * Get a list with all screen names that are defined
 * @return Screen names
 */
public static String[] getScreenNames() {
    return screens.keySet().toArray(new String[0]);
}

/**
 * Get the name of the current screen.
 * @return name of the screen or an exception in case it is not found
 */
public static String getCurrentScreenName() {
    for (String name : screens.keySet()) {
        Screen screen = screens.get(name);
        if (screen == getCurrentScreen()) {
```

```
            return name;
        }
    }
    throw new RuntimeException("Cannot get the current screen name. It seems like the current screen is not defined using the addScreen method.");
  }

  /**
   * Switches the currently active screen in the game to the one specified by name.
   *
   * @param name the name of the screen to switch to; must correspond to a registered screen
   * @throws IllegalStateException if the game has not been started yet.
   * @throws IllegalArgumentException if no screen with the specified name exists.
   */
  public static void switchScreen(String name) {
    if (gameInstance == null) {
      throw new IllegalStateException("GameApp not started yet. Call start() first.");
    }

    Screen screen = screens.get(name);
    if (screen == null) {
      throw new IllegalArgumentException("Screen '" + name + "' not found.");
    }

    // Dispose UI elements from the current screen if it has a HUD
    Screen current = gameInstance.getScreen();
    if (current instanceof HUD currentHUD && currentHUD.isHUDEnabled()) {
      disposeUIElements();
    }

    // Set the input processor automatically if the screen implements InputProcessor
    if (screen instanceof InputProcessor inputProcessor) {
      Gdx.input.setInputProcessor(inputProcessor);
    } else {
      Gdx.input.setInputProcessor(null);
    }

    gameInstance.setScreen(screen);
  }
```

```
  /**
   * Returns the currently active {@link Screen} of the game.
   *
   * @return the current screen, or {@code null} if the game has not been started yet
   */
  public static Screen getCurrentScreen() {
    if (gameInstance == null) return null;
    return gameInstance.getScreen();
  }

  /**
   * Returns the current width of the game window in pixels.
   * PLEASE NOTE: THIS IT NOT THE WORLD WIDTH
   * @return the width of the game window in pixels
   */
  public static int getWindowWidth() {
    return Gdx.graphics.getWidth();
  }

  /**
   * Returns the current height of the game window in pixels.
   * PLEASE NOTE: THIS IT NOT THE WORLD HEIGHT
   *
   * @return the height of the game window in pixels
   */
  public static int getWindowHeight() {
    return Gdx.graphics.getHeight();
  }

  /**
   * Get the width of the world (this method is identical to calling getWorldWidth() in your
own screen object)
   * @return width of the world
   */
  public static float getWorldWidth() {
    Screen currentScreen = getCurrentScreen();
    if (currentScreen instanceof WorldSize) {
      return ((WorldSize) currentScreen).getWorldWidth();
    } else {
      return getWindowWidth();
    }
```

```
    }

    /**
     * Get the height of the world (this method is identical to calling getWorldWidth() in
your own screen object)
     * @return height of the world
     */
    public static float getWorldHeight() {
        Screen currentScreen = getCurrentScreen();
        if (currentScreen instanceof WorldSize) {
            return ((WorldSize) currentScreen).getWorldHeight();
        } else {
            return getWindowHeight();
        }
    }

    /**
     * Returns the time elapsed since the last frame, in seconds.
     * <p>
     * This is equivalent to {@link Gdx#graphics}.getDeltaTime() in LibGDX and is useful
     * for frame rate independent calculations, such as movement or animations.
     * </p>
     *
     * @return the delta time in seconds since the last frame
     */
    public static float getDeltaTime() {
        return Gdx.graphics.getDeltaTime();
    }

    /**
     * Returns the current frame rate of the application, in frames per second (FPS).
     * <p>
     * This value is updated by LibGDX each frame and can be used for debugging,
     * performance monitoring, or display in an FPS counter.
     * </p>
     *
     * @return the number of frames rendered in the last second
     */
    public static int getFramesPerSecond() {
        return Gdx.graphics.getFramesPerSecond();
    }
```

```
    /**
     * WARNING: using this object can heavily influence the behaviour of the GameApp.
     * This is only useful for advanced users.
     *
     * @return The Game object from LibGDX, use to run this application.
     */
    public static Game getLibGDXGame() {
        return gameInstance;
    }

    //<editor-fold desc="Input handling">

    /**
     * Returns the current X-coordinate of the mouse cursor in screen coordinates.
     *
     * @return the X position of the mouse
     */
    public static int getMousePositionInWindowX() {
        return Gdx.input.getX();
    }

    /**
     * Returns the current Y-coordinate of the mouse cursor in screen coordinates.
     *
     * @return the Y position of the mouse
     */
    public static int getMousePositionInWindowY() {
        return Gdx.input.getY();
    }

    /**
     * Returns the current position of the mouse in screen coordinates as a {@link
Vector2}.
     *
     * @return a new Vector2 containing the mouse coordinates
     */
    public static Vector2 getMousePositionInWindow() {
        return new Vector2(getMousePositionInWindowX(), getMousePositionInWindowY());
    }
```

```
  /**
   * Returns true if the specified key is currently held down.
   *
   * @param keyCode the key code to check (see {@link com.badlogic.gdx.Input.Keys})
   * @return true if the key is pressed, false otherwise
   */
  public static boolean isKeyPressed(int keyCode) {
    return Gdx.input.isKeyPressed(keyCode);
  }

  /**
   * Returns true if the specified key was just pressed this frame.
   *
   * @param keyCode the key code to check (see {@link com.badlogic.gdx.Input.Keys})
   * @return true if the key was just pressed, false otherwise
   */
  public static boolean isKeyJustPressed(int keyCode) {
    return Gdx.input.isKeyJustPressed(keyCode);
  }

  /**
   * Returns true if the specified mouse button is currently held down.
   *
   * @param button the mouse button to check (see {@link
com.badlogic.gdx.Input.Buttons})
   * @return true if the button is pressed, false otherwise
   */
  public static boolean isButtonPressed(int button) {
    return Gdx.input.isButtonPressed(button);
  }

  /**
   * Returns true if the specified mouse button was just pressed this frame.
   *
   * @param button the mouse button to check (see {@link
com.badlogic.gdx.Input.Buttons})
   * @return true if the button was just pressed, false otherwise
   */
  public static boolean isButtonJustPressed(int button) {
    return Gdx.input.isButtonJustPressed(button);
  }
```

```
//</editor-fold>

//<editor-fold desc="Audio Methods">
/**
 * Loads and stores a sound effect from a file path under the given name.
 *
 * @param name     the name to associate with the sound
 * @param filePath the internal file path of the sound effect
 */
public static void addSound(String name, String filePath) {
   Sound sound = Gdx.audio.newSound(Gdx.files.internal(filePath));

   // In case there is already a sound with this name, dispose the old sound
   if (hasSound(name)) {
      disposeSound(name);
   }

   sounds.put(name.toLowerCase(), sound);
}

/**
 * Loads and stores a music track from a file path under the given name.
 *
 * @param name     the name to associate with the music
 * @param filePath the internal file path of the music track
 */
public static void addMusic(String name, String filePath) {
   Music music = Gdx.audio.newMusic(Gdx.files.internal(filePath));

   // In case there is already music with this name, dispose the old sound
   if (hasMusic(name)) {
      disposeMusic(name);
   }

   GameApp.music.put(name.toLowerCase(), music);
}

/**
 * Remove the sound and unload it from the memory.
 * @param name Name of the sound.
```

```
 */
public static void disposeSound(String name) {
   Sound s = getSound(name);
   s.stop();
   s.dispose();
}

/**
 * Remove the music and unload it from the memory.
 * @param name Name of the music.
 */
public static void disposeMusic(String name) {
   Music m = getMusic(name);
   m.stop();
   m.dispose();
}

/**
 * Plays a sound effect at the specified volume.
 *
 * @param name   the name of the loaded sound
 * @param volume the playback volume (0f - 1f)
 * @return the playback ID for the sound instance
 * @throws IllegalArgumentException if the sound has not been loaded
 */
public static long playSound(String name, float volume) {
   Sound sound = getSound(name);
   return sound.play(volume);
}

/**
 * Plays a sound effect at full volume (1.0).
 *
 * @param name the name of the loaded sound
 * @return the playback ID for the sound instance
 * @throws IllegalArgumentException if the sound has not been loaded
 */
public static long playSound(String name) {
   return playSound(name, 1f);
}
```

```
/**
 * Plays a music track, optionally looping.
 * <p>
 * If the music is already playing, this method does nothing.
 * </p>
 *
 * @param name    the name of the loaded music
 * @param looping {@code true} to loop the music, {@code false} to play once
 * @throws IllegalArgumentException if the music has not been loaded
 */
public static void playMusic(String name, boolean looping) {
   Music music = getMusic(name);
   if (!music.isPlaying()) {
      music.setLooping(looping);
      music.play();
   }
}

/**
 * Plays a music track, optionally looping with a specific volume.
 * <p>
 * If the music is already playing, this method does nothing.
 * </p>
 *
 * @param name    the name of the loaded music
 * @param looping {@code true} to loop the music, {@code false} to play once
 * @param volume the playback volume (0f - 1f)
 * @throws IllegalArgumentException if the music has not been loaded
 */
public static void playMusic(String name, boolean looping, float volume) {
   Music music = getMusic(name);
   if (!music.isPlaying()) {
      music.setVolume(volume);
      music.setLooping(looping);
      music.play();
   }
}

/**
 * Stops a specific sound effect by its playback ID.
 *
```

```
     * @param soundId the playback ID returned by {@link #playSound(String)}
     */
    public static void stopSound(long soundId) {
        for (Sound sound : sounds.values()) {
            sound.stop(soundId);
        }
    }

    /**
     * Stops all currently playing sound effects.
     */
    public static void stopAllSounds() {
        for (Sound sound : sounds.values()) {
            sound.stop();
        }
    }

    /**
     * Stops a specific music track by name.
     *
     * @param name the name of the loaded music track
     */
    public static void stopMusic(String name) {
        Music music = getMusic(name);
        if (music.isPlaying()) {
            music.stop();
        }
    }

    /**
     * Stops all currently playing music tracks.
     */
    public static void stopAllMusic() {
        for (Music music : music.values()) {
            if (music.isPlaying()) {
                music.stop();
            }
        }
    }

    /**
```

```
 * Stops all audio, including sound effects and music.
 */
public static void stopAllAudio() {
   stopAllSounds();
   stopAllMusic();
}

public static Sound getSound(String name) {
   if (sounds.containsKey(name.toLowerCase())) {
      return sounds.get(name.toLowerCase());
   } else {
      throw new IllegalArgumentException("Sound '" + name + "' not loaded.");
   }
}

public static Music getMusic(String name) {
   if (music.containsKey(name.toLowerCase())) {
      return music.get(name.toLowerCase());
   } else {
      throw new IllegalArgumentException("Music '" + name + "' not loaded.");
   }
}

/**
 * Returns if a sound is available with the given name
 * @param name Name of the sound
 * @return True, if it exists
 */
public static boolean hasSound(String name) {
   return sounds.containsKey(name.toLowerCase());
}

/**
 * Returns if music is available with the given name
 * @param name Name of the music
 * @return True, if it exists
 */
public static boolean hasMusic(String name) {
   return music.containsKey(name.toLowerCase());
}
```

```
  //</editor-fold>

  //<editor-fold desc="Drawing methods">
  /**
   * Returns the shared {@link ShapeRenderer} instance used for rendering shapes.
   *
   * @return the global {@code ShapeRenderer} instance
   */
  public static ShapeRenderer getShapeRenderer() {
     return shapeRenderer;
  }
  /**
   * Adds a named {@link Color} to the global color registry.
   *
   * @param name  the name to associate with the color
   * @param color the {@code Color} instance
   */
  public static void addColor(String name, Color color) {
     namedColors.put(name.toLowerCase(), color);
  }

  /**
   * Adds a named color to the global registry using RGBA components (0-255).
   *
   * @param name the name to associate with the color
   * @param r    red component (0-255)
   * @param g    green component (0-255)
   * @param b    blue component (0-255)
   * @param a    alpha component (0-255)
   */
  public static void addColor(String name, int r, int g, int b, int a) {
     Color color = new Color(r / 255.0f, g / 255.0f, b / 255.0f, a / 255.0f);
     addColor(name, color);
  }

  /**
   * Adds a named color to the global registry using RGB components (0-255) with full
opacity.
   *
   * @param name the name to associate with the color
   * @param r    red component (0-255)
```

```
     * @param g   green component (0-255)
     * @param b   blue component (0-255)
     */
    public static void addColor(String name, int r, int g, int b) {
        Color color = new Color(r / 255.0f, g / 255.0f, b / 255.0f, 1);
        addColor(name, color);
    }

    /**
     * Retrieves a named {@link Color} from the global color registry.
     *
     * @param name the name of the color
     * @return the {@code Color} associated with the name
     * @throws IllegalArgumentException if the color with the given name does not exist
     */
    public static Color getColor(String name) {
        Color color = namedColors.get(name.toLowerCase());
        if (color == null) {
            throw new IllegalArgumentException("Color '" + name + "' not found.");
        }
        return color;
    }

    /**
     * Get the names of all registered colors.
     * @return String array with all color names.
     */
    public static String[] getAllColors() {
        return namedColors.keySet().toArray(new String[0]);
    }

    /**
     * Checks whether a color with the given name exists in the global registry.
     *
     * @param name the name of the color
     * @return {@code true} if the color exists, {@code false} otherwise
     */
    public static boolean hasColor(String name) {
        return namedColors.containsKey(name.toLowerCase());
    }
```

```
/**
 * Begins shape rendering in filled mode.
 */
public static void startShapeRenderingFilled() {
   shapeRenderer.begin(ShapeRenderer.ShapeType.Filled);
}

/**
 * Begins shape rendering in outlined (line) mode.
 */
public static void startShapeRenderingOutlined() {
   shapeRenderer.begin(ShapeRenderer.ShapeType.Line);
}

/**
 * Begins shape rendering in point mode.
 */
public static void startShapeRenderingPoints() {
   shapeRenderer.begin(ShapeRenderer.ShapeType.Point);
}

/**
 * Set the width of the outline when drawing with the shape renderer.
 * @param width Width
 */
public static void setLineWidth(float width) {
   Gdx.gl.glLineWidth(width);
}

/**
 * Ends the current shape rendering batch.
 */
public static void endShapeRendering() {
   shapeRenderer.end();
}

/**
 * Clears the screen to the specified RGB color.
 *
 * @param r red component (0-255)
 * @param g green component (0-255)
```

```
     * @param b blue component (0-255)
     */
    public static void clearScreen(int r, int g, int b) {
        Gdx.gl.glClearColor(r / 255f, g / 255f, b / 255f, 1);
        Gdx.gl.glClear(GL20.GL_COLOR_BUFFER_BIT);
    }

    /**
     * Clears the screen to black (0, 0, 0).
     */
    public static void clearScreen() {
        clearScreen(0, 0, 0);
    }

    /**
     * Clears the screen using a named color from the color registry.
     *
     * @param colorName the name of the color to clear the screen with (see addColor
method)
     * @throws IllegalArgumentException if the color name is not found
     */
    public static void clearScreen(String colorName) {
        Color color = getColor(colorName);
        clearScreen(color);
    }

    /**
     * Clears the screen using the specified {@link Color}.
     *
     * @param color the color to clear the screen with
     */
    public static void clearScreen(Color color) {
        Gdx.gl.glClearColor(color.r, color.g, color.b, color.a);
        Gdx.gl.glClear(GL20.GL_COLOR_BUFFER_BIT);
    }

    /**
     * Sets the current drawing color for shapes.
     *
     * @param color the {@link Color} to use
     */
```

```
  public static void setColor(Color color) {
    checkMaskColor(color);
    shapeRenderer.setColor(color);
  }

  /**
   * Sets the current drawing color using RGB components (0-255) with full opacity.
   *
   * @param r red component (0-255)
   * @param g green component (0-255)
   * @param b blue component (0-255)
   */
  public static void setColor(int r, int g, int b) {
    Color color = new Color(r / 255f, g / 255f, b / 255f, 1f);
    checkMaskColor(color);
    shapeRenderer.setColor(color);
  }

  /**
   * Sets the current drawing color using RGBA components (0-255).
   *
   * @param r red component (0-255)
   * @param g green component (0-255)
   * @param b blue component (0-255)
   * @param a alpha component (0-255)
   */
  public static void setColor(int r, int g, int b, int a) {
    Color color = new Color(r / 255f, g / 255f, b / 255f, a / 255f);
    checkMaskColor(color);
    shapeRenderer.setColor(color);
  }

  /**
   * Sets the current drawing color using a registered color name.
   *
   * @param colorName the name of the color to use
   * @throws IllegalArgumentException if the color name is not found or invalid for
masking
   */
  public static void setColor(String colorName) {
    Color color = getColor(colorName);
```

```
    checkMaskColor(color);
    shapeRenderer.setColor(color);
}

/**
 * Checks whether the color is valid when a mask is active.
 * Only black or white is allowed when masking.
 *
 * @param color the color to check
 * @throws IllegalArgumentException if the color is invalid for masking
 */
private static void checkMaskColor(Color color) {
    if (maskActive) {
        if (!color.equals(Color.BLACK) && !color.equals(Color.WHITE)) {
            throw new IllegalArgumentException(
                "When a mask is active, only black or white can be drawn.");
        }
    }
}

/**
 * Enable transparency for shape render and sprite batch.
 */
public static void enableTransparency() {
    Gdx.gl.glEnable(GL20.GL_BLEND);
    Gdx.gl.glBlendFunc(GL20.GL_SRC_ALPHA, GL20.GL_ONE_MINUS_SRC_ALPHA);
}

/**
 * Disable transparency for shape render and sprite batch.
 */
public static void disableTransparency() {
    Gdx.gl.glDisable(GL20.GL_BLEND);
}

/**
 * Draws a rectangle.
 *
 * @param x      the x-coordinate of the bottom-left corner
 * @param y      the y-coordinate of the bottom-left corner
 * @param width  the width of the rectangle
```

```
 * @param height the height of the rectangle
 */
public static void drawRect(float x, float y, float width, float height) {
  shapeRenderer.rect(x, y, width, height);
}

/**
 * Draws a rectangle in a specific color.
 *
 * @param x       the x-coordinate of the bottom-left corner
 * @param y       the y-coordinate of the bottom-left corner
 * @param width   the width of the rectangle
 * @param height  the height of the rectangle
 * @param colorName the name of the color to use
 */
public static void drawRect(float x, float y, float width, float height, String colorName) {
  setColor(colorName);
  drawRect(x, y, width, height);
}

/**
 * Draws a rectangle in a specific color.
 *
 * @param x       the x-coordinate of the bottom-left corner
 * @param y       the y-coordinate of the bottom-left corner
 * @param width   the width of the rectangle
 * @param height  the height of the rectangle
 * @param color   the color object
 */
public static void drawRect(float x, float y, float width, float height, Color color) {
  setColor(color);
  drawRect(x, y, width, height);
}

/**
 * Draws a rectangle centered on a specified point.
 *
 * @param centerX the x-coordinate of the rectangle's center
 * @param centerY the y-coordinate of the rectangle's center
 * @param width   the width of the rectangle
 * @param height  the height of the rectangle
```

```
     */
    public static void drawRectCentered(float centerX, float centerY, float width, float
height) {
        float x = centerX - width / 2f;
        float y = centerY - height / 2f;
        shapeRenderer.rect(x, y, width, height);
    }

    /**
     * Draws a rectangle centered on a specified point with a color name.
     *
     * @param centerX   the x-coordinate of the rectangle's center
     * @param centerY   the y-coordinate of the rectangle's center
     * @param width     the width of the rectangle
     * @param height    the height of the rectangle
     * @param colorName the name of the color to use
     */
    public static void drawRectCentered(float centerX, float centerY, float width, float
height, String colorName) {
        setColor(colorName);
        drawRectCentered(centerX, centerY, width, height);
    }

    /**
     * Draws a rectangle centered on a specified point with a Color object.
     *
     * @param centerX the x-coordinate of the rectangle's center
     * @param centerY the y-coordinate of the rectangle's center
     * @param width   the width of the rectangle
     * @param height  the height of the rectangle
     * @param color   the color object to use
     */
    public static void drawRectCentered(float centerX, float centerY, float width, float
height, Color color) {
        setColor(color);
        drawRectCentered(centerX, centerY, width, height);
    }

    // --- Rounded Rectangles ---

    /**
```

```
    * Draws a rounded rectangle using the class's ShapeRenderer.
    * <p>
    * The rectangle consists of a central rectangle, side rectangles, and quarter-circle corners.
    * <strong>Note:</strong> This only works properly in filled drawing mode. Outline mode will not render correctly.
    * </p>
    *
    * @param x      the x-coordinate of the bottom-left corner
    * @param y      the y-coordinate of the bottom-left corner
    * @param width  the width of the rectangle
    * @param height the height of the rectangle
    * @param radius the radius of the rounded corners
    */
   public static void drawRoundedRect(float x, float y, float width, float height, float radius) {
       shapeRenderer.rect(x + radius, y, width - 2 * radius, height);
       shapeRenderer.rect(x, y + radius, radius, height - 2 * radius);
       shapeRenderer.rect(x + width - radius, y + radius, radius, height - 2 * radius);

       shapeRenderer.arc(x + radius, y + radius, radius, 180, 90);
       shapeRenderer.arc(x + width - radius, y + radius, radius, 270, 90);
       shapeRenderer.arc(x + width - radius, y + height - radius, radius, 0, 90);
       shapeRenderer.arc(x + radius, y + height - radius, radius, 90, 90);
   }

   /**
    * Draws a rounded rectangle with a specified color name.
    *
    * @param x       the x-coordinate of the bottom-left corner
    * @param y       the y-coordinate of the bottom-left corner
    * @param width     the width of the rectangle
    * @param height    the height of the rectangle
    * @param radius    the radius of the rounded corners
    * @param colorName the name of the color to use
    */
   public static void drawRoundedRect(float x, float y, float width, float height, float radius, String colorName) {
       setColor(colorName);
       drawRoundedRect(x, y, width, height, radius);
   }
```

```
  /**
   * Draws a rounded rectangle with a specified Color object.
   *
   * @param x     the x-coordinate of the bottom-left corner
   * @param y     the y-coordinate of the bottom-left corner
   * @param width  the width of the rectangle
   * @param height the height of the rectangle
   * @param radius the radius of the rounded corners
   * @param color  the Color object to use
   */
  public static void drawRoundedRect(float x, float y, float width, float height, float
radius, Color color) {
    setColor(color);
    drawRoundedRect(x, y, width, height, radius);
  }

  /**
   * Draws a rounded rectangle centered on a specified point.
   *
   * @param centerX the x-coordinate of the rectangle's center
   * @param centerY the y-coordinate of the rectangle's center
   * @param width   the width of the rectangle
   * @param height  the height of the rectangle
   * @param radius  the radius of the rounded corners
   */
  public static void drawRoundedRectCentered(float centerX, float centerY, float width,
float height, float radius) {
    float x = centerX - width / 2f;
    float y = centerY - height / 2f;
    drawRoundedRect(x, y, width, height, radius);
  }

  /**
   * Draws a rounded rectangle centered on a specified point with a color name.
   *
   * @param centerX   the x-coordinate of the rectangle's center
   * @param centerY   the y-coordinate of the rectangle's center
   * @param width     the width of the rectangle
   * @param height    the height of the rectangle
   * @param radius    the radius of the rounded corners
```

```
     * @param colorName the name of the color to use
     */
    public static void drawRoundedRectCentered(float centerX, float centerY, float width,
float height, float radius, String colorName) {
        setColor(colorName);
        drawRoundedRectCentered(centerX, centerY, width, height, radius);
    }

    /**
     * Draws a rounded rectangle centered on a specified point with a Color object.
     *
     * @param centerX the x-coordinate of the rectangle's center
     * @param centerY the y-coordinate of the rectangle's center
     * @param width   the width of the rectangle
     * @param height  the height of the rectangle
     * @param radius  the radius of the rounded corners
     * @param color   the Color object to use
     */
    public static void drawRoundedRectCentered(float centerX, float centerY, float width,
float height, float radius, Color color) {
        setColor(color);
        drawRoundedRectCentered(centerX, centerY, width, height, radius);
    }

    /**
     * Draws a circle.
     *
     * @param x      the x-coordinate of the center
     * @param y      the y-coordinate of the center
     * @param radius the radius of the circle
     */
    public static void drawCircle(float x, float y, float radius) {
        shapeRenderer.circle(x, y, radius);
    }

    /**
     * Draws a circle in a specific color.
     *
     * @param x       the x-coordinate of the center
     * @param y       the y-coordinate of the center
     * @param radius    the radius of the circle
```

```
 * @param colorName the name of the color to use
 */
public static void drawCircle(float x, float y, float radius, String colorName) {
   setColor(colorName);
   drawCircle(x, y, radius);
}

/**
 * Draws a circle in a specific color.
 *
 * @param x       the x-coordinate of the center
 * @param y       the y-coordinate of the center
 * @param radius    the radius of the circle
 * @param color     the color object
 */
public static void drawCircle(float x, float y, float radius, Color color) {
   setColor(color);
   drawCircle(x, y, radius);
}

/**
 * Draws a circle with a specified number of segments.
 *
 * @param x       the x-coordinate of the center
 * @param y       the y-coordinate of the center
 * @param radius   the radius of the circle
 * @param segments the number of segments used to approximate the circle
 */
public static void drawCircle(float x, float y, float radius, int segments) {
   shapeRenderer.circle(x, y, radius, segments);
}

/**
 * Draws a circle in a specific color with a specified number of segments.
 *
 * @param x       the x-coordinate of the center
 * @param y       the y-coordinate of the center
 * @param radius    the radius of the circle
 * @param segments  the number of segments used to approximate the circle
 * @param colorName the name of the color to use
 */
```

```
  public static void drawCircle(float x, float y, float radius, int segments, String
colorName) {
    setColor(colorName);
    drawCircle(x, y, radius, segments);
  }

  /**
   * Draws a circle in a specific color with a specified number of segments.
   *
   * @param x       the x-coordinate of the center
   * @param y       the y-coordinate of the center
   * @param radius    the radius of the circle
   * @param segments  the number of segments used to approximate the circle
   * @param color     the color object
   */
  public static void drawCircle(float x, float y, float radius, int segments, Color color) {
    setColor(color);
    drawCircle(x, y, radius, segments);
  }

  /**
   * Draws an ellipse.
   *
   * @param x     the x-coordinate of the bottom-left corner
   * @param y     the y-coordinate of the bottom-left corner
   * @param width  the width of the ellipse
   * @param height the height of the ellipse
   */
  public static void drawEllipse(float x, float y, float width, float height) {
    shapeRenderer.ellipse(x, y, width, height);
  }

  /**
   * Draws an ellipse in a specific color.
   *
   * @param x       the x-coordinate of the bottom-left corner
   * @param y       the y-coordinate of the bottom-left corner
   * @param width     the width of the ellipse
   * @param height    the height of the ellipse
   * @param colorName the name of the color to use
   */
```

```
  public static void drawEllipse(float x, float y, float width, float height, String
colorName) {
    setColor(colorName);
    drawEllipse(x, y, width, height);
  }

  /**
   * Draws an ellipse in a specific color.
   *
   * @param x       the x-coordinate of the bottom-left corner
   * @param y       the y-coordinate of the bottom-left corner
   * @param width    the width of the ellipse
   * @param height   the height of the ellipse
   * @param color    the color object
   */
  public static void drawEllipse(float x, float y, float width, float height, Color color) {
    setColor(color);
    drawEllipse(x, y, width, height);
  }

  /**
   * Draws an ellipse centered on a specified point.
   *
   * @param centerX the x-coordinate of the ellipse's center
   * @param centerY the y-coordinate of the ellipse's center
   * @param width   the width of the ellipse
   * @param height  the height of the ellipse
   */
  public static void drawEllipseCentered(float centerX, float centerY, float width, float
height) {
    float x = centerX - width / 2f;
    float y = centerY - height / 2f;
    shapeRenderer.ellipse(x, y, width, height);
  }

  /**
   * Draws an ellipse centered on a specified point with a color name.
   *
   * @param centerX   the x-coordinate of the ellipse's center
   * @param centerY   the y-coordinate of the ellipse's center
   * @param width     the width of the ellipse
```

```
     * @param height    the height of the ellipse
     * @param colorName the name of the color to use
     */
    public static void drawEllipseCentered(float centerX, float centerY, float width, float
height, String colorName) {
        setColor(colorName);
        drawEllipseCentered(centerX, centerY, width, height);
    }

    /**
     * Draws an ellipse centered on a specified point with a Color object.
     *
     * @param centerX the x-coordinate of the ellipse's center
     * @param centerY the y-coordinate of the ellipse's center
     * @param width   the width of the ellipse
     * @param height  the height of the ellipse
     * @param color   the color object to use
     */
    public static void drawEllipseCentered(float centerX, float centerY, float width, float
height, Color color) {
        setColor(color);
        drawEllipseCentered(centerX, centerY, width, height);
    }

    /**
     * Draws a line.
     *
     * @param x1 the x-coordinate of the start point
     * @param y1 the y-coordinate of the start point
     * @param x2 the x-coordinate of the end point
     * @param y2 the y-coordinate of the end point
     */
    public static void drawLine(float x1, float y1, float x2, float y2) {
        shapeRenderer.line(x1, y1, x2, y2);
    }

    /**
     * Draws a line in a specific color.
     *
     * @param x1       the x-coordinate of the start point
     * @param y1       the y-coordinate of the start point
```

```
     * @param x2       the x-coordinate of the end point
     * @param y2       the y-coordinate of the end point
     * @param colorName the name of the color to use
     */
    public static void drawLine(float x1, float y1, float x2, float y2, String colorName) {
        setColor(colorName);
        drawLine(x1, y1, x2, y2);
    }

    /**
     * Draws a line in a specific color.
     *
     * @param x1       the x-coordinate of the start point
     * @param y1       the y-coordinate of the start point
     * @param x2       the x-coordinate of the end point
     * @param y2       the y-coordinate of the end point
     * @param color    the color object
     */
    public static void drawLine(float x1, float y1, float x2, float y2, Color color) {
        setColor(color);
        drawLine(x1, y1, x2, y2);
    }

    /**
     * Draws a polygon.
     *
     * @param vertices the array of vertices {x1, y1, x2, y2, ...}
     */
    public static void drawPolygon(float[] vertices) {
        // Check the current ShapeRenderer type
        ShapeRenderer.ShapeType type = shapeRenderer.getCurrentType();

        if (type == ShapeRenderer.ShapeType.Filled) {
            // Fill polygon using triangulation
            EarClippingTriangulator triangulator = new EarClippingTriangulator();
            ShortArray triangles = triangulator.computeTriangles(vertices);

            for (int i = 0; i < triangles.size; i += 3) {
                int p1 = triangles.get(i) * 2;
                int p2 = triangles.get(i + 1) * 2;
                int p3 = triangles.get(i + 2) * 2;
```

```
            shapeRenderer.triangle(
                vertices[p1], vertices[p1 + 1],
                vertices[p2], vertices[p2 + 1],
                vertices[p3], vertices[p3 + 1]
            );
        }
    } else {
        // Draw outline polygon
        shapeRenderer.polygon(vertices);
    }
}

/**
 * Draws a polygon in a specific color.
 *
 * @param vertices  the array of vertices {x1, y1, x2, y2, ...}
 * @param colorName the name of the color to use
 */
public static void drawPolygon(float[] vertices, String colorName) {
    setColor(colorName);
    drawPolygon(vertices);
}

/**
 * Draws a polygon in a specific color.
 *
 * @param vertices  the array of vertices {x1, y1, x2, y2, ...}
 * @param color     the color object
 */
public static void drawPolygon(float[] vertices, Color color) {
    setColor(color);
    drawPolygon(vertices);
}

//</editor-fold>

//<editor-fold desc="Texture methods">
/**
 * Returns the shared {@link SpriteBatch} instance used for rendering sprites.
 *
```

```
     * @return the global {@code SpriteBatch} instance
     */
    public static SpriteBatch getSpriteBatch() {
        return spriteBatch;
    }

    /**
     * Begins sprite rendering.
     *
     * @throws IllegalStateException if called while a mask is active
     */
    public static void startSpriteRendering() {
        if (maskActive) {
            throw new IllegalStateException("Cannot render sprites when we are still creating
a mask.");
        }
        spriteBatch.begin();
    }

    /**
     * Ends the current sprite rendering batch.
     */
    public static void endSpriteRendering() {
        spriteBatch.end();
    }

    /**
     * Loads a texture from a file path and stores it in the global texture registry.
     *
     * @param name     the name to associate with the texture
     * @param filePath the internal file path of the texture image
     */
    public static void addTexture(String name, String filePath) {
        addTexture(name, filePath, Texture.TextureFilter.Linear, Texture.TextureFilter.Linear);
    }

    /**
     * Loads a texture from a file path and stores it in the global texture registry.
     *
     * @param name     the name to associate with the texture
     * @param filePath the internal file path of the texture image
```

```
     * @param filter   the quality filter for the texture (when scaling)
     */
    public static void addTexture(String name, String filePath, Texture.TextureFilter filter) {
        addTexture(name, filePath, filter, filter);
    }

    /**
     * Loads a texture from a file path and stores it in the global texture registry.
     *
     * @param name     the name to associate with the texture
     * @param filePath the internal file path of the texture image
     * @param minFilter the filter used when scaling down the texture
     * @param maxFilter the filter used when scaling up the texture
     */
    public static void addTexture(String name, String filePath, Texture.TextureFilter
minFilter, Texture.TextureFilter maxFilter) {
        Texture texture = new Texture(Gdx.files.internal(filePath));
        texture.setFilter(minFilter, maxFilter);

        // If there is already a texture with this name, dispose it
        if (hasTexture(name.toLowerCase())) {
            Texture t = textures.remove(name.toLowerCase());
            t.dispose();
        }

        textures.put(name.toLowerCase(), texture);
    }

    /**
     * Retrieves a texture from the global texture registry by name.
     *
     * @param name the name of the texture
     * @return the {@link Texture} associated with the name
     * @throws IllegalArgumentException if the texture has not been loaded
     */
    public static Texture getTexture(String name) {
        Texture texture = textures.get(name.toLowerCase());
        if (texture == null) {
            throw new IllegalArgumentException("Texture '" + name + "' not found.");
        }
        return texture;
```

```
}

/**
 * Returns the width in pixels of the texture with the given name.
 *
 * @param name the name or identifier of the texture
 * @return the width of the texture in pixels
 * @throws IllegalArgumentException if no texture is found for the given name
 */
public static int getTextureWidth(String name) {
   return getTexture(name).getWidth();
}

/**
 * Returns the height in pixels of the texture with the given name.
 *
 * @param name the name or identifier of the texture
 * @return the height of the texture in pixels
 * @throws IllegalArgumentException if no texture is found for the given name
 */
public static int getTextureHeight(String name) {
   return getTexture(name).getHeight();
}

/**
 * Returns if texture is available with the given name
 * @param name Name of the texture
 * @return True, if it exists
 */
public static boolean hasTexture(String name) {
   return textures.containsKey(name.toLowerCase());
}

/**
 * Remove the texture and unload it from the memory.
 * @param name Name of the texture
 */
public static void disposeTexture(String name) {
   // Needs to be fixed (problem with render() and hide()
   //      getTexture(name.toLowerCase()).dispose();
}
```

```
  /**
   * Draws a texture at the specified position using the current sprite batch.
   *
   * @param name the name of the loaded texture
   * @param x    the x-coordinate of the bottom-left corner
   * @param y    the y-coordinate of the bottom-left corner
   * @throws IllegalArgumentException if the texture has not been loaded
   */
  public static void drawTexture(String name, float x, float y) {
    spriteBatch.draw(getTexture(name), x, y);
  }

  /**
   * Draws a texture at the specified position and size using the current sprite batch.
   *
   * @param name   the name of the loaded texture
   * @param x      the x-coordinate of the bottom-left corner
   * @param y      the y-coordinate of the bottom-left corner
   * @param width  the width to draw the texture
   * @param height the height to draw the texture
   * @throws IllegalArgumentException if the texture has not been loaded
   */
  public static void drawTexture(String name, float x, float y, float width, float height) {
    spriteBatch.draw(getTexture(name), x, y, width, height);
  }

  /**
   * Draws a texture at the specified position, size, rotation, and flipping using the current
sprite batch.
   * <p>
   * The rotation is performed around the center of the drawn texture. Flip parameters
allow mirroring
   * horizontally or vertically.
   * </p>
   *
   * @param name         the name of the loaded texture
   * @param x            the x-coordinate of the bottom-left corner
   * @param y            the y-coordinate of the bottom-left corner
   * @param width        the width to draw the texture
   * @param height       the height to draw the texture
```

```
     * @param rotationDegrees the rotation in degrees (clockwise)
     * @param flipX          whether to flip the texture horizontally
     * @param flipY          whether to flip the texture vertically
     * @throws IllegalArgumentException if the texture has not been loaded
     */
    public static void drawTexture(String name, float x, float y, float width, float height,
float rotationDegrees, boolean flipX, boolean flipY) {
        Texture texture = getTexture(name);

        float originX = width / 2f;
        float originY = height / 2f;

        spriteBatch.draw(
            texture,
            x, y,
            originX, originY,
            width, height,
            1f, 1f, // scaleX, scaleY
            rotationDegrees,
            0, 0, // srcX, srcY
            texture.getWidth(),
            texture.getHeight(),
            flipX, flipY
        );
    }

    /**
     * Draws a texture centered at the specified position using the current sprite batch.
     *
     * @param name the name of the loaded texture
     * @param centerX the x-coordinate of the center
     * @param centerY the y-coordinate of the center
     * @throws IllegalArgumentException if the texture has not been loaded
     */
    public static void drawTextureCentered(String name, float centerX, float centerY) {
        Texture texture = getTexture(name);
        float x = centerX - texture.getWidth() / 2f;
        float y = centerY - texture.getHeight() / 2f;
        spriteBatch.draw(texture, x, y);
    }
```

```
    /**
     * Draws a texture centered at the specified position with size using the current sprite batch.
     *
     * @param name   the name of the loaded texture
     * @param centerX the x-coordinate of the center
     * @param centerY the y-coordinate of the center
     * @param width  the width to draw the texture
     * @param height the height to draw the texture
     * @throws IllegalArgumentException if the texture has not been loaded
     */
    public static void drawTextureCentered(String name, float centerX, float centerY, float width, float height) {
        float x = centerX - width / 2f;
        float y = centerY - height / 2f;
        spriteBatch.draw(getTexture(name), x, y, width, height);
    }

    /**
     * Draws a texture centered at the specified position with size, rotation, and flipping using the current sprite batch.
     * <p>
     * Rotation is around the center of the texture.
     * </p>
     *
     * @param name          the name of the loaded texture
     * @param centerX       the x-coordinate of the center
     * @param centerY       the y-coordinate of the center
     * @param width         the width to draw the texture
     * @param height        the height to draw the texture
     * @param rotationDegrees the rotation in degrees (clockwise)
     * @param flipX         whether to flip the texture horizontally
     * @param flipY         whether to flip the texture vertically
     * @throws IllegalArgumentException if the texture has not been loaded
     */
    public static void drawTextureCentered(String name, float centerX, float centerY, float width, float height, float rotationDegrees, boolean flipX, boolean flipY) {
        Texture texture = getTexture(name);

        float x = centerX - width / 2f;
        float y = centerY - height / 2f;
```

```
        float originX = width / 2f;
        float originY = height / 2f;

        spriteBatch.draw(
            texture,
            x, y,
            originX, originY,
            width, height,
            1f, 1f, // scaleX, scaleY
            rotationDegrees,
            0, 0, // srcX, srcY
            texture.getWidth(),
            texture.getHeight(),
            flipX, flipY
        );
    }
    //</editor-fold>

    //<editor-fold desc="Text drawing">
    /**
     * Loads a TTF or OTF font, optionally scaling it to world height or keeping pixel size.
     *
     * @param name       Name to reference the font
     * @param filePath   Path to the TTF/OTF file in assets
     * @param size       Font size in pixels
     * @param usePixelSize True to use literal pixel size (good for HUD), false to scale to world height
     */
    public static void addFont(String name, String filePath, float size, boolean usePixelSize) {
        FreeTypeFontGenerator generator = new FreeTypeFontGenerator(Gdx.files.internal(filePath));
        FreeTypeFontGenerator.FreeTypeFontParameter parameter = new FreeTypeFontGenerator.FreeTypeFontParameter();

        if (!usePixelSize && getCurrentScreen() instanceof WorldSize) {
            float worldHeight = ((WorldSize) getCurrentScreen()).getWorldHeight();
            parameter.size = (int) (size * (Gdx.graphics.getHeight() / worldHeight));
        } else {
            parameter.size = (int) (size * Gdx.graphics.getDensity());
        }
```

```
    parameter.color = Color.WHITE;
    parameter.genMipMaps = true;
    parameter.minFilter = Texture.TextureFilter.MipMapLinearLinear;
    parameter.magFilter = Texture.TextureFilter.Linear;

    BitmapFont font = generator.generateFont(parameter);
    generator.dispose();

    registerFont(name, font);
}

/**
 * Overload: Adds a font that scales to world height by default.
 *
 * @param name     Name to reference the font
 * @param filePath Path to the TTF/OTF file
 * @param size     Font size in pixels
 */
public static void addFont(String name, String filePath, float size) {
    addFont(name, filePath, size, false);
}

/**
 * Registers an already created {@link BitmapFont} under a given name.
 *
 * @param name Name to reference the font
 * @param font Pre-created BitmapFont
 */
public static void addFont(String name, BitmapFont font) {
    registerFont(name, font);
}

/** Registers the font, disposing the old one if necessary */
private static void registerFont(String name, BitmapFont font) {
    if (fonts.containsKey(name.toLowerCase())) {
        BitmapFont fontToDispose = getFont(name.toLowerCase());
        fontToDispose.dispose();
        fonts.remove(name.toLowerCase());
    }
    fonts.put(name.toLowerCase(), font);
```

```
    }

    /**
     * Loads a styled TTF font with optional border and shadow, with world-scaling or pixel-perfect option.
     *
     * @param name        Name to reference the font
     * @param filePath    Path to the TTF/OTF file
     * @param size        Font size in pixels
     * @param color       Base color
     * @param borderWidth  Width of border (0 = no border)
     * @param borderColor  Color of border (ignored if borderWidth = 0)
     * @param shadowOffsetX Shadow X offset
     * @param shadowOffsetY Shadow Y offset
     * @param shadowColor   Shadow color
     * @param usePixelSize  True to use pixel size (HUD), false to scale to world height
     */
    public static void addStyledFont(
        String name,
        String filePath,
        int size,
        Color color,
        float borderWidth,
        Color borderColor,
        int shadowOffsetX,
        int shadowOffsetY,
        Color shadowColor,
        boolean usePixelSize
    ) {
        FreeTypeFontGenerator generator = new FreeTypeFontGenerator(Gdx.files.internal(filePath));
        FreeTypeFontGenerator.FreeTypeFontParameter parameter = new FreeTypeFontGenerator.FreeTypeFontParameter();

        if (!usePixelSize && getCurrentScreen() instanceof WorldSize) {
            float worldHeight = ((WorldSize) getCurrentScreen()).getWorldHeight();
            parameter.size = (int) (size * (Gdx.graphics.getHeight() / worldHeight));
        } else {
            parameter.size = (int) (size * Gdx.graphics.getDensity());
        }
```

```
    parameter.color = color;
    parameter.borderWidth = borderWidth;
    parameter.borderColor = borderColor != null ? borderColor : Color.BLACK;
    parameter.shadowOffsetX = shadowOffsetX;
    parameter.shadowOffsetY = shadowOffsetY;
    parameter.shadowColor = shadowColor != null ? shadowColor : Color.BLACK;

    BitmapFont font = generator.generateFont(parameter);
    generator.dispose();

    registerFont(name, font);
}

/**
 * Styled font with named colors (throws if color names not found).
 *
 * @param name          Name to reference the font
 * @param filePath      TTF/OTF file path
 * @param size          Font size
 * @param colorName     Base color name
 * @param borderWidth   Border width
 * @param borderColorName Border color name
 * @param shadowOffsetX  Shadow X offset
 * @param shadowOffsetY  Shadow Y offset
 * @param shadowColorName Shadow color name
 * @param usePixelSize   True = pixel size, false = world-scaled
 */
public static void addStyledFont(
    String name,
    String filePath,
    int size,
    String colorName,
    float borderWidth,
    String borderColorName,
    int shadowOffsetX,
    int shadowOffsetY,
    String shadowColorName,
    boolean usePixelSize
) {
    Color color = GameApp.getColor(colorName);
    Color borderColor = GameApp.getColor(borderColorName);
```

```
        Color shadowColor = GameApp.getColor(shadowColorName);

        addStyledFont(name, filePath, size, color, borderWidth, borderColor,
shadowOffsetX, shadowOffsetY, shadowColor, usePixelSize);
    }

    /**
     * Retrieves a loaded font by its name.
     *
     * @param name Name of the loaded font.
     * @return The BitmapFont associated with the given name.
     * @throws IllegalArgumentException if the font is not found.
     */
    public static BitmapFont getFont(String name) {
        BitmapFont font = fonts.get(name.toLowerCase());
        if (font == null) throw new IllegalArgumentException("Font '" + name + "' not found.");
        return font;
    }

    /**
     * Returns if font is available with the given name
     * @param name Name of the font
     * @return True, if it exists
     */
    public static boolean hasFont(String name) {
        return fonts.containsKey(name.toLowerCase());
    }

    /**
     * Remove the font and unload it from the memory.
     * @param name Name of the font.
     */
    public static void disposeFont(String name) {
// For now we don't dispose the font (because of render() and hide() problem)
//      BitmapFont font = getFont(name.toLowerCase());
//      font.dispose();
//      fonts.remove(name.toLowerCase());
    }

    /**
     * Draws text left-bottom aligned at the specified position with a given color.
```

```
     *
     * @param fontName Name of the loaded font.
     * @param text Text to draw.
     * @param x X-coordinate of the bottom-left corner of the text.
     * @param y Y-coordinate of the bottom-left corner of the text.
     * @param color Color to render the text.
     */
    public static void drawText(String fontName, String text, float x, float y, Color color) {
        BitmapFont font = getFont(fontName);
        font.setColor(color);

        // Measure the text with GlyphLayout
        GlyphLayout layout = new GlyphLayout(font, text);

        // baselineY = y + layout.height ensures bottom of text aligns with y
        float baselineY = y + layout.height;

        // Draw the text
        font.draw(spriteBatch, layout, x, baselineY);
    }

    /**
     * Draws text left-bottom aligned at the specified position with a given color.
     *
     * @param fontName Name of the loaded font.
     * @param text Text to draw.
     * @param x X-coordinate of the bottom-left corner of the text.
     * @param y Y-coordinate of the bottom-left corner of the text.
     * @param color Color to render the text.
     */
    public static void drawText(String fontName, String text, float x, float y, String color) {
        drawText(fontName, text, x, y, getColor(color));
    }

    /**
     * Draws text centered on a specified point with a given color.
     *
     * @param fontName Name of the loaded font.
     * @param text Text to draw.
     * @param centerX X-coordinate of the center of the text.
     * @param centerY Y-coordinate of the center of the text.
```

```
     * @param color Color to render the text.
     */
    public static void drawTextCentered(String fontName, String text, float centerX, float
centerY, Color color) {
        BitmapFont font = getFont(fontName);
        font.setColor(color);
        GlyphLayout layout = new GlyphLayout(font, text);
        float x = centerX - layout.width / 2f;
        float y = centerY + layout.height / 2f;
        font.draw(spriteBatch, text, x, y);
    }

    /**
     * Draws text centered on a specified point with a given color.
     *
     * @param fontName Name of the loaded font.
     * @param text Text to draw.
     * @param centerX X-coordinate of the center of the text.
     * @param centerY Y-coordinate of the center of the text.
     * @param color Color to render the text.
     */
    public static void drawTextCentered(String fontName, String text, float centerX, float
centerY, String color) {
        drawTextCentered(fontName, text, centerX, centerY, getColor(color));
    }

    /**
     * Draws text horizontally centered at a given X-coordinate with a given Y-coordinate
and color.
     *
     * @param fontName Name of the loaded font.
     * @param text Text to draw.
     * @param centerX X-coordinate of the horizontal center of the text.
     * @param y Y-coordinate for the text baseline.
     * @param color Color to render the text.
     */
    public static void drawTextHorizontallyCentered(String fontName, String text, float
centerX, float y, Color color) {
        BitmapFont font = getFont(fontName);
        font.setColor(color);
        GlyphLayout layout = new GlyphLayout(font, text);
```

```
        float x = centerX - layout.width / 2f;
        font.draw(spriteBatch, text, x, y);
    }

    /**
     * Overload with color as string.
     */
    public static void drawTextHorizontallyCentered(String fontName, String text, float centerX, float y, String color) {
        drawTextHorizontallyCentered(fontName, text, centerX, y, getColor(color));
    }

    /**
     * Draws text vertically centered at a given Y-coordinate with a given X-coordinate and color.
     *
     * @param fontName Name of the loaded font.
     * @param text Text to draw.
     * @param x X-coordinate for the left of the text.
     * @param centerY Y-coordinate of the vertical center of the text.
     * @param color Color to render the text.
     */
    public static void drawTextVerticallyCentered(String fontName, String text, float x, float centerY, Color color) {
        BitmapFont font = getFont(fontName);
        font.setColor(color);
        GlyphLayout layout = new GlyphLayout(font, text);
        float y = centerY + layout.height / 2f;
        font.draw(spriteBatch, text, x, y);
    }

    /**
     * Overload with color as string.
     */
    public static void drawTextVerticallyCentered(String fontName, String text, float x, float centerY, String color) {
        drawTextVerticallyCentered(fontName, text, x, centerY, getColor(color));
    }

    /**
     * Returns the width of a given text in pixels when rendered with the specified font.
```

```
     *
     * @param fontName Name of the loaded font.
     * @param text Text to measure.
     * @return Width of the text in pixels.
     */
    public static float getTextWidth(String fontName, String text) {
        BitmapFont font = getFont(fontName);
        return new GlyphLayout(font, text).width;
    }

    /**
     * Returns the height of a given text in pixels when rendered with the specified font.
     *
     * @param fontName Name of the loaded font.
     * @param text Text to measure.
     * @return Height of the text in pixels.
     */
    public static float getTextHeight(String fontName, String text) {
        BitmapFont font = getFont(fontName);
        return new GlyphLayout(font, text).height;
    }
    //</editor-fold>

    //<editor-fold desc="Mathematical methods">
    /**
     * Clamps a value to be within the specified minimum and maximum range.
     *
     * @param value the value to clamp
     * @param min   the minimum allowable value
     * @param max   the maximum allowable value
     * @return the clamped value; if {@code value} is less than {@code min}, returns
{@code min},
     *       if {@code value} is greater than {@code max}, returns {@code max},
     *       otherwise returns {@code value} itself
     */
    public static float clamp(float value, float min, float max) {
        return MathUtils.clamp(value, min, max);
    }

    /**
     * Returns a random float value within the specified range. The minimim is inclusive,
```

```
the maximum is exclusive.
     *
     * @param min the lower bound
     * @param max the upper bound
     * @return a random float between {@code min} and {@code max}
     */
    public static float random(float min, float max) {
        return MathUtils.random(min, max);
    }

    /**
     * Returns a random int value within the specified range. The minimim is inclusive, the
maximum is exclusive.
     *
     * @param min the lower bound
     * @param max the upper bound
     * @return a random int between {@code min} and {@code max}
     */
    public static int randomInt(int min, int max) {
        return MathUtils.random(min, max-1);
    }

    /**
     * Returns a random element from an array.
     *
     * @param array the array to pick from
     * @param <T> type of elements
     * @return a random element from the array
     * @throws IllegalArgumentException if the array is null or empty
     */
    public static <T> T random(T[] array) {
        if (array == null || array.length == 0) {
            throw new IllegalArgumentException("Array must not be null or empty");
        }
        int index = MathUtils.random(array.length - 1);
        return array[index];
    }

    /**
     * Returns a random element from a List of any type.
     *
```

```
     * @param list the list to pick from
     * @param <T> the type of elements
     * @return a random element from the list
     * @throws IllegalArgumentException if the list is null or empty
     */
    public static <T> T random(List<T> list) {
        if (list == null || list.isEmpty()) {
            throw new IllegalArgumentException("List must not be null or empty");
        }
        int index = MathUtils.random(list.size() - 1);
        return list.get(index);
    }

    /**
     * Checks if two axis-aligned rectangles overlap.
     *
     * @param x1 X of first rectangle
     * @param y1 Y of first rectangle
     * @param w1 Width of first rectangle
     * @param h1 Height of first rectangle
     * @param x2 X of second rectangle
     * @param y2 Y of second rectangle
     * @param w2 Width of second rectangle
     * @param h2 Height of second rectangle
     * @return true if the rectangles intersect
     */
    public static boolean rectOverlap(float x1, float y1, float w1, float h1,
                                      float x2, float y2, float w2, float h2) {
        Rectangle r1 = new Rectangle(x1, y1, w1, h1);
        Rectangle r2 = new Rectangle(x2, y2, w2, h2);
        return r1.overlaps(r2);
    }

    public static boolean rectOverlap(Rectangle r1, Rectangle r2) {
        return r1.overlaps(r2);
    }

    /**
     * Checks if two circles overlap.
     *
     * @param x1 X center of first circle
```

```
     * @param y1 Y center of first circle
     * @param r1 Radius of first circle
     * @param x2 X center of second circle
     * @param y2 Y center of second circle
     * @param r2 Radius of second circle
     * @return true if the circles intersect
     */
    public static boolean circleOverlap(float x1, float y1, float r1,
                                        float x2, float y2, float r2) {
        Circle c1 = new Circle(x1, y1, r1);
        Circle c2 = new Circle(x2, y2, r2);
        return Intersector.overlaps(c1, c2);
    }

    public static boolean circleOverlap(Circle c1, Circle c2) {
        return Intersector.overlaps(c1, c2);
    }

    /**
     * Checks if an axis-aligned rectangle and a circle overlap.
     *
     * @param rectX X of rectangle
     * @param rectY Y of rectangle
     * @param rectW Width of rectangle
     * @param rectH Height of rectangle
     * @param circleX X center of circle
     * @param circleY Y center of circle
     * @param radius Radius of circle
     * @return true if the rectangle and circle intersect
     */
    public static boolean rectCircleOverlap(float rectX, float rectY, float rectW, float rectH,
                                            float circleX, float circleY, float radius) {
        Rectangle rect = new Rectangle(rectX, rectY, rectW, rectH);
        Circle circle = new Circle(circleX, circleY, radius);
        return Intersector.overlaps(circle, rect);
    }

    public static boolean rectCircleOverlap(Rectangle rect, Circle circle) {
        return Intersector.overlaps(circle, rect);
    }
```

```
/**
 * Checks if a point is inside a rectangle.
 *
 * @param px X coordinate of the point
 * @param py Y coordinate of the point
 * @param rx X of rectangle
 * @param ry Y of rectangle
 * @param rw Width of rectangle
 * @param rh Height of rectangle
 * @return true if the point is inside the rectangle
 */
public static boolean pointInRect(float px, float py, float rx, float ry, float rw, float rh) {
   return px >= rx && px <= rx + rw && py >= ry && py <= ry + rh;
}

public static boolean pointInRect(Vector2 point, Rectangle rect) {
   return rect.contains(point);
}

/**
 * Checks if a point is inside a circle.
 *
 * @param px X coordinate of the point
 * @param py Y coordinate of the point
 * @param cx X center of circle
 * @param cy Y center of circle
 * @param radius Radius of the circle
 * @return true if the point is inside the circle
 */
public static boolean pointInCircle(float px, float py, float cx, float cy, float radius) {
   Circle circle = new Circle(cx, cy, radius);
   return circle.contains(px, py);
}

public static boolean pointInCircle(Vector2 point, Circle circle) {
   return circle.contains(point);
}

/**
 * Checks if one rectangle is completely contained within another rectangle.
 *
```

```
 * @param x1 X of outer rectangle
 * @param y1 Y of outer rectangle
 * @param w1 Width of outer rectangle
 * @param h1 Height of outer rectangle
 * @param x2 X of inner rectangle
 * @param y2 Y of inner rectangle
 * @param w2 Width of inner rectangle
 * @param h2 Height of inner rectangle
 * @return true if the inner rectangle is fully inside the outer rectangle
 */
public static boolean rectContainsRect(float x1, float y1, float w1, float h1,
                                       float x2, float y2, float w2, float h2) {
   Rectangle outer = new Rectangle(x1, y1, w1, h1);
   Rectangle inner = new Rectangle(x2, y2, w2, h2);
   return outer.contains(inner);
}

public static boolean rectContainsRect(Rectangle outer, Rectangle inner) {
   return outer.contains(inner);
}

/**
 * Calculates the distance between two points.
 *
 * @param x1 X of first point
 * @param y1 Y of first point
 * @param x2 X of second point
 * @param y2 Y of second point
 * @return distance between the points
 */
public static float distance(float x1, float y1, float x2, float y2) {
   return new Vector2(x1, y1).dst(x2, y2);
}

/**
 * Calculates the angle in degrees from the first point to the second point.
 * 0 degrees points to the right (positive X axis), angles increase counter-clockwise.
 *
 * @param x1 X of first point
 * @param y1 Y of first point
 * @param x2 X of second point
```

```
     * @param y2 Y of second point
     * @return angle in degrees from the first point to the second point
     */
    public static float angleBetween(float x1, float y1, float x2, float y2) {
        return new Vector2(x2, y2).sub(x1, y1).angleDeg();
    }

    /**
     * Computes new coordinates by moving from a starting point in a specified direction and distance.
     *
     * <p>The direction is in degrees, where 0° points to the right (positive X-axis) and
     * increases counterclockwise: 90° is up, 180° is left, 270° is down.</p>
     *
     * @param x the starting x-coordinate
     * @param y the starting y-coordinate
     * @param distance the distance to move (can also be thought of as speed)
     * @param directionDegrees the movement direction in degrees
     * @return a float array of length 2, where index 0 is the new x-coordinate and index 1 is the new y-coordinate
     */
    public static float[] moveInDirection(float x, float y, float distance, float directionDegrees) {
        float radians = directionDegrees * MathUtils.degreesToRadians;
        float newX = x + distance * MathUtils.cos(radians);
        float newY = y + distance * MathUtils.sin(radians);
        return new float[] {newX, newY};
    }
    //</editor-fold>

    //<editor-fold desc="Store and serialization">

    /**
     * Add item to the application store, so that it can be used in multiple screens.
     * @param name Key in the store
     * @param anyType Object to store
     */
    public static void addItemToStore(String name, Object anyType) {
        store.put(name.toLowerCase(), anyType);
    }
```

```
/**
 * Check if an item exists with the given name
 * @param name Name of the item
 * @return True, in case the item exists.
 */
public static boolean hasItemInStore(String name) {
   return store.containsKey(name.toLowerCase());
}

/**
 * Get an item from the application store by name.
 * @param name Name of the item in the store
 * @return Object, if available. Otherwise, exception.
 */
public static Object getItemFromStore(String name) {
   if (store.containsKey(name.toLowerCase())) {
      return store.get(name.toLowerCase());
   } else {
      throw new IllegalArgumentException(name + " not found in the store.");
   }
}

/**
 * Remove an item from the store by name.
 *
 * @param name Name of the item
 * @return true if the item existed and was removed, false otherwise
 */
public static boolean removeItemFromStore(String name) {
   return store.remove(name.toLowerCase()) != null;
}

/**
 * Saves any object to a binary file.
 * @param object the object to serialize
 * @param fileName the file name (e.g., "savegame.bin")
 */
public static void saveToBinary(Object object, String fileName) {
   Kryo kryo = new Kryo();
   FileHandle file = Gdx.files.local(fileName);
   Output output = new Output(file.write(false));
```

```
    kryo.writeClassAndObject(output, object);
    output.close();
}

/**
 * Loads any object from a binary file.
 * @param fileName the file name
 * @return the deserialized object, or null if file doesn't exist
 */
public static Object loadFromBinary(String fileName) {
    Kryo kryo = new Kryo();
    FileHandle file = Gdx.files.local(fileName);
    if (!file.exists()) {
        throw new IllegalArgumentException(fileName + " does not exist.");
    }

    // Read file and return object.
    try (Input input = new Input(file.read())) {
        return kryo.readClassAndObject(input);
    }
}

/**
 * Checks if the save file exists.
 * @param fileName filename
 * @return true, if the file exists
 */
public static boolean fileExists(String fileName) {
    FileHandle file = Gdx.files.local(fileName);
    return file.exists();
}

/**
 * Serialize any object to JSON and save to a file.
 *
 * @param object   the object to serialize
 * @param fileName the file name (e.g., "savegame.json")
 */
public static void saveToJson(Object object, String fileName) {
    Json json = new Json();
    String jsonString = json.toJson(object);
```

```
        Gdx.files.local(fileName).writeString(jsonString, false);
    }

    /**
     * Load an object from a JSON file.
     *
     * @param fileName  the file name
     * @param type      the class type of the object to load
     * @param <T>       the type parameter
     * @return the deserialized object
     * @throws IllegalArgumentException if the file doesn't exist
     */
    public static <T> T loadFromJson(String fileName, Class<T> type) {
        FileHandle file = Gdx.files.local(fileName);
        if (!file.exists()) {
            throw new IllegalArgumentException(fileName + " does not exist.");
        }
        Json json = new Json();
        return json.fromJson(type, file.readString());
    }

    //</editor-fold>

    //<editor-fold desc="Interpolation/tweening">
    /**
     * Creates a new interpolator with the given name.
     * If an interpolator with the same name exists, it will be replaced.
     *
     * @param name     the unique name of the interpolator
     * @param start    the starting value
     * @param end      the ending value
     * @param duration the duration of the interpolation in seconds
     * @param strategy the interpolation strategy (e.g., {@link Interpolation#pow2Out},
{@link Interpolation#bounce})
     */
    public static void addInterpolator(String name, float start, float end, float duration,
Interpolation strategy) {
        // Remove the old interpolator
        if (hasInterpolator(name)) {
            disposeInterpolator(name);
        }
```

```
        // Create a new one
        interpolators.put(name.toLowerCase(), new Interpolator(start, end, duration,
strategy));
    }

    /**
     * Creates a new interpolator with the given name.
     * If an interpolator with the same name exists, it will be replaced.
     *
     * @param name     the unique name of the interpolator
     * @param start    the starting value
     * @param end      the ending value
     * @param duration the duration of the interpolation in seconds
     * @param strategy the interpolation strategy (e.g., "bounce", "elastic", "pow1")
     */
    public static void addInterpolator(String name, float start, float end, float duration,
String strategy) {
        // Remove the old interpolator
        if (hasInterpolator(name)) {
            disposeInterpolator(name);
        }
        // Create a new one
        interpolators.put(name.toLowerCase(), new Interpolator(start, end, duration,
strategy));
    }

    /**
     * Get a list of all the possible interpolation strategies available
     * @return String[] with all possible interpolation strategies.
     */
    public static String[] getInterpolationStrategies() {
        return Interpolator.getAllInterpolationNames();
    }

    /**
     * Removes an interpolator from the internal registry.
     * After calling this, the interpolator with the given name will no longer exist.
     *
     * @param name the name of the interpolator to remove (case-insensitive)
     */
    public static void disposeInterpolator(String name) {
```

```
    interpolators.remove(name.toLowerCase());
  }

  /**
   * Checks whether an interpolator with the given name exists.
   *
   * @param name the name of the interpolator to check (case-insensitive)
   * @return {@code true} if the interpolator exists, {@code false} otherwise
   */
  public static boolean hasInterpolator(String name) {
    return interpolators.containsKey(name.toLowerCase());
  }

  /**
   * Retrieves an existing interpolator by name.
   * Use this if you need to access the interpolator object directly.
   *
   * @param name the name of the interpolator to retrieve (case-insensitive)
   * @return the {@link Interpolator} instance, or {@code null} if it does not exist
   */
  public static Interpolator getInterpolator(String name) {
    return interpolators.get(name.toLowerCase());
  }

  /**
   * Updates the interpolator with the given name and returns the current value.
   * <p>
   * This method should be called every frame (e.g., inside the render loop).
   *
   * @param name the name of the interpolator
   * @return the current interpolated value
   * @throws IllegalArgumentException if no interpolator exists with the given name
   */
  public static float updateInterpolator(String name) {
    Interpolator i = interpolators.get(name.toLowerCase());
    if (i == null) throw new IllegalArgumentException("No interpolator with name: " +
name);
    return i.update(Gdx.graphics.getDeltaTime());
  }

  /**
```

```
    * Checks whether the interpolator has finished its animation.
    *
    * @param name the name of the interpolator
    * @return true if the interpolation has completed, false otherwise
    * @throws IllegalArgumentException if no interpolator exists with the given name
    */
   public static boolean isInterpolatorFinished(String name) {
      Interpolator i = interpolators.get(name.toLowerCase());
      if (i == null) throw new IllegalArgumentException("No interpolator with name: " +
name);
      return i.isFinished();
   }

   /**
    * Resets the interpolator with the given name to start its animation from the beginning.
    * If the interpolator does not exist, this method does nothing.
    *
    * @param name the name of the interpolator
    */
   public static void resetInterpolator(String name) {
      resetInterpolator(name, false);
   }

   /**
    * Resets the interpolator with the given name to start its animation from the beginning.
    * If the interpolator does not exist, this method does nothing.
    *
    * @param name the name of the interpolator
    * @param reverse should the animation be reversed?
    */
   public static void resetInterpolator(String name, boolean reverse) {
      Interpolator i = interpolators.get(name.toLowerCase());
      if (i != null) i.reset(reverse);
   }
   //</editor-fold>

   //<editor-fold desc="Sprite sheets">
   /**
    * Loads a spritesheet from a file, slices it into frames, and stores the frames.
    * The underlying Texture is not added to the global texture map.
    *
```

```
     * @param name       the name to identify the spritesheet
     * @param filePath   the internal file path of the texture
     * @param frameWidth  the width of a single frame
     * @param frameHeight the height of a single frame
     */
    public static void addSpriteSheet(String name, String filePath, int frameWidth, int
frameHeight) {
        // Load the texture locally (not stored in the textures map)
        Texture texture = new Texture(Gdx.files.internal(filePath));

        // Slice into TextureRegion[][] and store in the spritesheet map
        TextureRegion[][] frames = TextureRegion.split(texture, frameWidth, frameHeight);
        spritesheets.put(name.toLowerCase(), frames);
    }

    /**
     * Retrieve a specific frame from a spritesheet.
     *
     * @param name the spritesheet name
     * @param row  the row index (0-based)
     * @param col  the column index (0-based)
     * @return the TextureRegion for the requested frame
     * @throws IllegalArgumentException  if the spritesheet does not exist
     * @throws IndexOutOfBoundsException if row/col are invalid
     */
    public static TextureRegion getSpritesheetFrame(String name, int row, int col) {
        TextureRegion[][] frames = spritesheets.get(name.toLowerCase());
        if (frames == null) throw new IllegalArgumentException("Spritesheet not found: " +
name);
        if (row < 0 || row >= frames.length)
            throw new IndexOutOfBoundsException("Row out of bounds: " + row);
        if (col < 0 || col >= frames[row].length)
            throw new IndexOutOfBoundsException("Col out of bounds: " + col);
        return frames[row][col];
    }

    /**
     * Draw a specific frame from a spritesheet at the given position.
     *
     * @param name the spritesheet name
     * @param row  the row index (0-based)
```

```
     * @param col  the column index (0-based)
     * @param x    the x-coordinate of the bottom-left corner
     * @param y    the y-coordinate of the bottom-left corner
     */
    public static void drawSpritesheetFrame(String name, int row, int col, float x, float y) {
        TextureRegion frame = getSpritesheetFrame(name, row, col);
        spriteBatch.draw(frame, x, y);
    }

    /**
     * Draw a specific frame with scaling.
     *
     * @param name   the spritesheet name
     * @param row    the row index (0-based)
     * @param col    the column index (0-based)
     * @param x      the x-coordinate
     * @param y      the y-coordinate
     * @param width  the width to draw
     * @param height the height to draw
     */
    public static void drawSpritesheetFrame(String name, int row, int col, float x, float y,
float width, float height) {
        TextureRegion frame = getSpritesheetFrame(name, row, col);
        spriteBatch.draw(frame, x, y, width, height);
    }

    /**
     * Draw a specific frame with scaling, rotation, and flipping.
     *
     * @param name         the spritesheet name
     * @param row          the row index (0-based)
     * @param col          the column index (0-based)
     * @param x            the x-coordinate
     * @param y            the y-coordinate
     * @param width        the width to draw
     * @param height       the height to draw
     * @param rotationDegrees rotation in degrees (clockwise)
     * @param flipX        whether to flip horizontally
     * @param flipY        whether to flip vertically
     */
    public static void drawSpritesheetFrame(String name, int row, int col, float x, float y,
```

```
float width, float height,
                            float rotationDegrees, boolean flipX, boolean flipY) {
    TextureRegion frame = getSpritesheetFrame(name, row, col);
    float originX = width / 2f;
    float originY = height / 2f;

    spriteBatch.draw(
      frame.getTexture(),
      x, y,
      originX, originY,
      width, height,
      1f, 1f,
      rotationDegrees,
      frame.getRegionX(),
      frame.getRegionY(),
      frame.getRegionWidth(),
      frame.getRegionHeight(),
      flipX, flipY
    );
  }

  /**
   * Draw a frame centered at a given position.
   *
   * @param name    the spritesheet name
   * @param row     the row index (0-based)
   * @param col     the column index (0-based)
   * @param centerX the x-coordinate of the center
   * @param centerY the y-coordinate of the center
   */
  public static void drawSpritesheetFrameCentered(String name, int row, int col, float
centerX, float centerY) {
    TextureRegion frame = getSpritesheetFrame(name, row, col);
    float x = centerX - frame.getRegionWidth() / 2f;
    float y = centerY - frame.getRegionHeight() / 2f;
    spriteBatch.draw(frame, x, y);
  }

  /**
   * Draw a frame centered with scaling.
   *
```

```
    * @param name    the spritesheet name
    * @param row     the row index (0-based)
    * @param col     the column index (0-based)
    * @param centerX the x-coordinate of the center
    * @param centerY the y-coordinate of the center
    * @param width   the width to draw
    * @param height  the height to draw
    */
   public static void drawSpritesheetFrameCentered(String name, int row, int col, float
centerX, float centerY,
                                   float width, float height) {
      float x = centerX - width / 2f;
      float y = centerY - height / 2f;
      drawSpritesheetFrame(name, row, col, x, y, width, height);
   }

   /**
    * Draw a frame centered with scaling, rotation, and flipping.
    *
    * @param name          the spritesheet name
    * @param row           the row index (0-based)
    * @param col           the column index (0-based)
    * @param centerX       the x-coordinate of the center
    * @param centerY       the y-coordinate of the center
    * @param width         the width to draw
    * @param height        the height to draw
    * @param rotationDegrees rotation in degrees (clockwise)
    * @param flipX         whether to flip horizontally
    * @param flipY         whether to flip vertically
    */
   public static void drawSpritesheetFrameCentered(String name, int row, int col, float
centerX, float centerY,
                                   float width, float height, float rotationDegrees,
                                   boolean flipX, boolean flipY) {
      TextureRegion frame = getSpritesheetFrame(name, row, col);
      float x = centerX - width / 2f;
      float y = centerY - height / 2f;
      float originX = width / 2f;
      float originY = height / 2f;

      spriteBatch.draw(
```

```
            frame.getTexture(),
            x, y,
            originX, originY,
            width, height,
            1f, 1f,
            rotationDegrees,
            frame.getRegionX(),
            frame.getRegionY(),
            frame.getRegionWidth(),
            frame.getRegionHeight(),
            flipX, flipY
        );
    }

    /**
     * Returns the number of rows in the specified spritesheet.
     *
     * @param name the name of the spritesheet
     * @return the number of rows
     * @throws IllegalArgumentException if the spritesheet does not exist
     */
    public static int getSpritesheetNumberOfRows(String name) {
        TextureRegion[][] frames = spritesheets.get(name.toLowerCase());
        if (frames == null) throw new IllegalArgumentException("Spritesheet not found: " +
name);
        return frames.length;
    }

    /**
     * Returns the number of columns in the specified spritesheet.
     * <p>
     * Assumes all rows have the same number of columns.
     *
     * @param name the name of the spritesheet
     * @return the number of columns
     * @throws IllegalArgumentException if the spritesheet does not exist
     */
    public static int getSpritesheetNumberOfCols(String name) {
        TextureRegion[][] frames = spritesheets.get(name.toLowerCase());
        if (frames == null) throw new IllegalArgumentException("Spritesheet not found: " +
name);
```

```
    if (frames.length == 0) return 0;
    return frames[0].length;
}

/**
 * Returns if a spritesheet is available with the given name.
 *
 * @param name the name of the spritesheet
 * @return true if it exists, false otherwise
 */
public static boolean hasSpritesheet(String name) {
    return spritesheets.containsKey(name.toLowerCase());
}

/**
 * Removes the spritesheet and disposes its underlying Texture.
 *
 * @param name the name of the spritesheet
 */
public static void disposeSpritesheet(String name) {
    TextureRegion[][] frames = spritesheets.remove(name.toLowerCase());
    if (frames != null) {
        frames[0][0].getTexture().dispose(); // disposes the underlying texture
    }
}
//</editor-fold>

//<editor-fold desc="Texture atlases">
/**
 * Loads a texture atlas and stores it in the internal atlas map.
 *
 * @param name     the name to identify the atlas
 * @param filePath the path to the .atlas file
 */
public static void addTextureAtlas(String name, String filePath) {
    if (!atlases.containsKey(name.toLowerCase())) {
        TextureAtlas atlas = new TextureAtlas(Gdx.files.internal(filePath));
        atlases.put(name.toLowerCase(), atlas);
    }
}
```

```
  /**
   * Returns if an atlas is available with the given name
   *
   * @param name the name of the atlas
   * @return true if it exists, false otherwise
   */
  public static boolean hasAtlas(String name) {
     return atlases.containsKey(name.toLowerCase());
  }

  /**
   * Removes the atlas and disposes it from memory.
   *
   * @param name the name of the atlas
   */
  public static void disposeAtlas(String name) {
     String key = name.toLowerCase();
     TextureAtlas atlas = atlases.remove(key);
     if (atlas != null) {
        atlas.dispose();
     }
  }

  /**
   * Retrieves a {@link TextureRegion} from a loaded texture atlas.
   * <p>
   * This method automatically supports both single-region and indexed multi-region
atlases.
   * If the region name ends with an underscore followed by a number (e.g. "walk_1"),
   * the method will interpret it as an indexed region and fetch the correct frame from
   * the corresponding atlas sequence.
   * </p>
   *
   * <p>For example:</p>
   * <pre>
   * getAtlasRegion("characters", "walk_0"); // retrieves the first frame
   * getAtlasRegion("characters", "walk_1"); // retrieves the second frame
   * getAtlasRegion("characters", "idle");   // retrieves a single non-indexed region
   * </pre>
   *
   * @param atlasName  the name of the atlas
```

```
    * @param regionName the name of the region inside the atlas (optionally ending with
_N)
    * @return the {@link TextureRegion} for the requested region
    * @throws IllegalArgumentException if the atlas or region does not exist
    */
   public static TextureRegion getAtlasRegion(String atlasName, String regionName) {
      TextureAtlas atlas = atlases.get(atlasName.toLowerCase());
      if (atlas == null) {
         throw new IllegalArgumentException("Atlas not found: " + atlasName);
      }

      // Check if the name ends with _<number>, e.g., "walk_3"
      int underscoreIndex = regionName.lastIndexOf('_');
      if (underscoreIndex != -1 && underscoreIndex < regionName.length() - 1) {
         String prefix = regionName.substring(0, underscoreIndex);
         String suffix = regionName.substring(underscoreIndex + 1);

         try {
            int index = Integer.parseInt(suffix);
            Array<TextureAtlas.AtlasRegion> regions = atlas.findRegions(prefix);

            if (regions.isEmpty()) {
               throw new IllegalArgumentException("No regions found with prefix '" + prefix +
"' in atlas: " + atlasName);
            }
            if (index < 0 || index >= regions.size) {
               throw new IndexOutOfBoundsException("Invalid region index " + index + " for '"
+ prefix + "'");
            }

            return regions.get(index);
         } catch (NumberFormatException ignored) {
            // Not actually an index, fall through to single-region lookup
         }
      }

      // Fallback: simple single-region lookup
      TextureAtlas.AtlasRegion region = atlas.findRegion(regionName);
      if (region == null) {
         throw new IllegalArgumentException("Region '" + regionName + "' not found in
atlas: " + atlasName);
```

```
        }

        return region;
    }

    /**
     * Get the texture atlas object.
     * @param name Name of the texture atlas
     * @return Texture atlas object
     */
    public static TextureAtlas getAtlas(String name) {
        TextureAtlas atlas = atlases.get(name.toLowerCase());
        if (atlas == null) throw new IllegalArgumentException("Atlas not found: " + name);
        return atlas;
    }

    /**
     * Draws a region from an atlas at the specified position.
     *
     * @param atlasName  the name of the atlas
     * @param regionName the name of the region
     * @param x          the x-coordinate of the bottom-left corner
     * @param y          the y-coordinate of the bottom-left corner
     */
    public static void drawAtlasRegion(String atlasName, String regionName, float x, float
y) {
        TextureRegion region = getAtlasRegion(atlasName, regionName);
        spriteBatch.draw(region, x, y);
    }

    /**
     * Draws a region from an atlas at the specified position with scaling.
     *
     * @param atlasName  the name of the atlas
     * @param regionName the name of the region
     * @param x          the x-coordinate
     * @param y          the y-coordinate
     * @param width      the width to draw
     * @param height     the height to draw
     */
    public static void drawAtlasRegion(String atlasName, String regionName, float x, float
```

```
y, float width, float height) {
    TextureRegion region = getAtlasRegion(atlasName, regionName);
    spriteBatch.draw(region, x, y, width, height);
  }

  /**
   * Draws a region from an atlas at the specified position with scaling, rotation, and
flipping.
   *
   * @param atlasName      the name of the atlas
   * @param regionName     the name of the region
   * @param x              the x-coordinate
   * @param y              the y-coordinate
   * @param width          the width to draw
   * @param height         the height to draw
   * @param rotationDegrees rotation in degrees (clockwise)
   * @param flipX          whether to flip horizontally
   * @param flipY          whether to flip vertically
   */
  public static void drawAtlasRegion(String atlasName, String regionName, float x, float
y, float width, float height,
                                     float rotationDegrees, boolean flipX, boolean flipY) {
    TextureRegion region = getAtlasRegion(atlasName, regionName);
    float originX = width / 2f;
    float originY = height / 2f;

    spriteBatch.draw(
      region.getTexture(),
      x, y,
      originX, originY,
      width, height,
      1f, 1f,
      rotationDegrees,
      region.getRegionX(),
      region.getRegionY(),
      region.getRegionWidth(),
      region.getRegionHeight(),
      flipX, flipY
    );
  }
```

```
    /**
     * Draws a region from an atlas centered at a given position.
     *
     * @param atlasName  the name of the atlas
     * @param regionName the name of the region
     * @param centerX    the x-coordinate of the center
     * @param centerY    the y-coordinate of the center
     */
    public static void drawAtlasRegionCentered(String atlasName, String regionName,
float centerX, float centerY) {
        TextureRegion region = getAtlasRegion(atlasName, regionName);
        float x = centerX - region.getRegionWidth() / 2f;
        float y = centerY - region.getRegionHeight() / 2f;
        spriteBatch.draw(region, x, y);
    }

    /**
     * Draws a region from an atlas centered with scaling.
     *
     * @param atlasName  the name of the atlas
     * @param regionName the name of the region
     * @param centerX    the x-coordinate of the center
     * @param centerY    the y-coordinate of the center
     * @param width      the width to draw
     * @param height     the height to draw
     */
    public static void drawAtlasRegionCentered(String atlasName, String regionName,
float centerX, float centerY, float width, float height) {
        float x = centerX - width / 2f;
        float y = centerY - height / 2f;
        drawAtlasRegion(atlasName, regionName, x, y, width, height);
    }

    /**
     * Draws a region from an atlas centered with scaling, rotation, and flipping.
     *
     * @param atlasName      the name of the atlas
     * @param regionName     the name of the region
     * @param centerX        the x-coordinate of the center
     * @param centerY        the y-coordinate of the center
     * @param width          the width to draw
```

```
     * @param height        the height to draw
     * @param rotationDegrees rotation in degrees (clockwise)
     * @param flipX         whether to flip horizontally
     * @param flipY         whether to flip vertically
     */
    public static void drawAtlasRegionCentered(String atlasName, String regionName,
float centerX, float centerY,
                               float width, float height, float rotationDegrees,
                               boolean flipX, boolean flipY) {
        TextureRegion region = getAtlasRegion(atlasName, regionName);
        float x = centerX - width / 2f;
        float y = centerY - height / 2f;
        float originX = width / 2f;
        float originY = height / 2f;

        spriteBatch.draw(
            region.getTexture(),
            x, y,
            originX, originY,
            width, height,
            1f, 1f,
            rotationDegrees,
            region.getRegionX(),
            region.getRegionY(),
            region.getRegionWidth(),
            region.getRegionHeight(),
            flipX, flipY
        );
    }

    //</editor-fold>

    //<editor-fold desc="Animations">

    /**
     * Creates an empty animation with a given frame duration and loop flag.
     * Frames can be added manually later using addAnimationFrame or
addAnimationFrameFromAtlas.
     *
     * @param name          the unique name of the animation
     * @param frameDuration the duration of each frame in seconds
```

```
     * @param loop        whether the animation should loop
     */
    public static void addEmptyAnimation(String name, float frameDuration, boolean
loop) {
        if (hasAnimation(name)) {
            disposeAnimation(name);
        }

        SpriteAnimation anim = new SpriteAnimation(frameDuration, loop);
        animations.put(name.toLowerCase(), anim);
    }

    /**
     * Adds a new animation using all frames from the specified spritesheet.
     * Frames are taken row by row, left to right.
     * Replaces any existing animation with the same name.
     *
     * @param name        the unique animation name
     * @param spritesheet   the name of the spritesheet
     * @param frameDuration seconds per frame
     * @param loop        whether the animation should loop
     * @throws IllegalArgumentException if the spritesheet does not exist
     */
    public static void addAnimationFromSpritesheet(String name, String spritesheet, float
frameDuration, boolean loop) {
        if (hasAnimation(name)) disposeAnimation(name);

        TextureRegion[][] sheetFrames = spritesheets.get(spritesheet.toLowerCase());
        if (sheetFrames == null)
            throw new IllegalArgumentException("Spritesheet not found: " + spritesheet);

        // Flatten 2D array into 1D array row by row
        int rows = sheetFrames.length;
        int cols = sheetFrames[0].length;
        TextureRegion[] frames = new TextureRegion[rows * cols];
        int index = 0;
        for (int r = 0; r < rows; r++) {
            for (int c = 0; c < cols; c++) {
                frames[index++] = sheetFrames[r][c];
            }
        }
```

```
    animations.put(name.toLowerCase(), new SpriteAnimation(frames, frameDuration,
loop));
  }

  /**
   * Adds a new animation using manually specified frames.
   * Replaces any existing animation with the same name.
   *
   * @param name        the unique animation name
   * @param frames      the array of frames to use
   * @param frameDuration seconds per frame
   * @param loop        whether the animation should loop
   */
  public static void addAnimation(String name, TextureRegion[] frames, float
frameDuration, boolean loop) {
    if (hasAnimation(name)) disposeAnimation(name);
    animations.put(name.toLowerCase(), new SpriteAnimation(frames, frameDuration,
loop));
  }

  /**
   * Adds a single frame from a spritesheet (by row and column) to an existing animation.
   * <p>
   * The animation **must already exist**. Use {@link #addEmptyAnimation(String, float,
boolean)}
   * or one of the other add-animation methods to create it first.
   * </p>
   *
   * @param animationName the name of the animation (must already exist)
   * @param spritesheet   the name of the spritesheet
   * @param row         the row index (0-based)
   * @param col         the column index (0-based)
   * @throws IllegalArgumentException  if the spritesheet or animation does not exist
   * @throws IndexOutOfBoundsException if row/col are out of bounds
   */
  public static void addAnimationFrameFromSpritesheet(String animationName, String
spritesheet, int row, int col) {
    TextureRegion[][] sheetFrames = spritesheets.get(spritesheet.toLowerCase());
    if (sheetFrames == null) {
      throw new IllegalArgumentException("Spritesheet not found: " + spritesheet);
```

```
    }

    if (row < 0 || row >= sheetFrames.length) {
      throw new IndexOutOfBoundsException("Row out of bounds: " + row);
    }
    if (col < 0 || col >= sheetFrames[row].length) {
      throw new IndexOutOfBoundsException("Col out of bounds: " + col);
    }

    TextureRegion frame = sheetFrames[row][col];

    SpriteAnimation anim = animations.get(animationName.toLowerCase());
    if (anim == null) {
      throw new IllegalArgumentException("Animation not found: " + animationName + ".
Create it first with addEmptyAnimation(...) or addAnimation(...).");
    }

    anim.addFrame(frame);
  }

  /**
   * Creates a new animation using all regions in a given atlas that match a prefix.
   * Regions are automatically sorted alphabetically.
   *
   * @param name        the name of the animation
   * @param atlasName   the name of the loaded atlas
   * @param prefix      the prefix of the region names to include (e.g., "run_")
   * @param frameDuration duration of each frame in seconds
   * @param loop        whether the animation should loop
   */
  public static void addAnimationFromAtlas(String name, String atlasName, String
prefix, float frameDuration, boolean loop) {
    if (hasAnimation(name)) disposeAnimation(name);

    TextureAtlas atlas = atlases.get(atlasName.toLowerCase());
    if (atlas == null) throw new IllegalArgumentException("Atlas not found: " +
atlasName);

    Array<TextureAtlas.AtlasRegion> regions = atlas.findRegions(prefix);
    if (regions.isEmpty()) throw new IllegalArgumentException(
      "No regions found with prefix '" + prefix + "' in atlas: " + atlasName);
```

```
        SpriteAnimation anim = new SpriteAnimation(frameDuration, loop);
        for (TextureAtlas.AtlasRegion region : regions) {
            anim.addFrame(region);
        }

        animations.put(name.toLowerCase(), anim);
    }

    /**
     * Adds a single region from an atlas to an existing animation.
     *
     * @param animationName the name of the animation
     * @param atlasName     the name of the atlas
     * @param regionName    the specific region name to add
     */
    public static void addAnimationFrameFromAtlas(String animationName, String
atlasName, String regionName) {
        SpriteAnimation anim = animations.get(animationName.toLowerCase());
        if (anim == null) throw new IllegalArgumentException("Animation not found: " +
animationName);

        TextureRegion region = getAtlasRegion(atlasName, regionName);
        anim.addFrame(region);
    }

    /**
     * Checks if an animation exists.
     *
     * @param name the animation name
     * @return true if it exists
     */
    public static boolean hasAnimation(String name) {
        return animations.containsKey(name.toLowerCase());
    }

    /**
     * Removes an animation.
     *
     * @param name the animation name
     */
```

```
  public static void disposeAnimation(String name) {
    animations.remove(name.toLowerCase());
  }

  /**
   * Retrieves an animation by name.
   *
   * @param name the animation name
   * @return the SpriteAnimation instance, or null if it does not exist
   */
  public static SpriteAnimation getAnimation(String name) {
    return animations.get(name.toLowerCase());
  }

  /**
   * Updates the animation by the frame's delta time.
   * <p>
   * Students do not need to handle {@link TextureRegion} directly; drawing
   * methods will automatically use the current frame.
   *
   * @param name the name of the animation
   * @throws IllegalArgumentException if no animation exists with the given name
   */
  public static void updateAnimation(String name) {
    SpriteAnimation anim = animations.get(name.toLowerCase());
    if (anim == null) throw new IllegalArgumentException("No animation with name: " +
name);
    anim.update(Gdx.graphics.getDeltaTime());
  }

  /**
   * Draws the current frame of the animation at the specified bottom-left position.
   *
   * @param name the name of the animation
   * @param x    the x-coordinate of the bottom-left corner
   * @param y    the y-coordinate of the bottom-left corner
   * @throws IllegalArgumentException if no animation exists with the given name
   */
  public static void drawAnimation(String name, float x, float y) {
    SpriteAnimation anim = animations.get(name.toLowerCase());
    if (anim == null) throw new IllegalArgumentException("No animation with name: " +
```

```
name);
    spriteBatch.draw(anim.getCurrentFrame(), x, y);
  }

  /**
   * Draws the current frame of the animation at the specified position and size.
   *
   * @param name   the name of the animation
   * @param x      the x-coordinate of the bottom-left corner
   * @param y      the y-coordinate of the bottom-left corner
   * @param width  the width to draw
   * @param height the height to draw
   * @throws IllegalArgumentException if no animation exists with the given name
   */
  public static void drawAnimation(String name, float x, float y, float width, float height) {
    SpriteAnimation anim = animations.get(name.toLowerCase());
    if (anim == null) throw new IllegalArgumentException("No animation with name: " +
name);
    spriteBatch.draw(anim.getCurrentFrame(), x, y, width, height);
  }

  /**
   * Draws the current frame of the animation at the specified position, size, rotation, and
flipping.
   *
   * @param name         the name of the animation
   * @param x            the x-coordinate of the bottom-left corner
   * @param y            the y-coordinate of the bottom-left corner
   * @param width        the width to draw
   * @param height       the height to draw
   * @param rotationDegrees rotation in degrees (clockwise)
   * @param flipX        whether to flip the frame horizontally
   * @param flipY        whether to flip the frame vertically
   * @throws IllegalArgumentException if no animation exists with the given name
   */
  public static void drawAnimation(String name, float x, float y, float width, float height,
                                   float rotationDegrees, boolean flipX, boolean flipY) {
    SpriteAnimation anim = animations.get(name.toLowerCase());
    if (anim == null) throw new IllegalArgumentException("No animation with name: " +
name);
```

```
        TextureRegion frame = anim.getCurrentFrame();
        float originX = width / 2f;
        float originY = height / 2f;

        spriteBatch.draw(
            frame.getTexture(),
            x, y,
            originX, originY,
            width, height,
            1f, 1f,
            rotationDegrees,
            frame.getRegionX(),
            frame.getRegionY(),
            frame.getRegionWidth(),
            frame.getRegionHeight(),
            flipX, flipY
        );
    }

    /**
     * Draws the current frame of the animation centered at the specified position.
     *
     * @param name    the name of the animation
     * @param centerX the x-coordinate of the center
     * @param centerY the y-coordinate of the center
     * @throws IllegalArgumentException if no animation exists with the given name
     */
    public static void drawAnimationCentered(String name, float centerX, float centerY) {
        SpriteAnimation anim = animations.get(name.toLowerCase());
        if (anim == null) throw new IllegalArgumentException("No animation with name: " +
name);

        TextureRegion frame = anim.getCurrentFrame();
        float x = centerX - frame.getRegionWidth() / 2f;
        float y = centerY - frame.getRegionHeight() / 2f;
        spriteBatch.draw(frame, x, y);
    }

    /**
     * Draws the current frame of the animation centered at the specified position with the
given size.
```

```
     *
     * @param name    the name of the animation
     * @param centerX the x-coordinate of the center
     * @param centerY the y-coordinate of the center
     * @param width   the width to draw
     * @param height  the height to draw
     * @throws IllegalArgumentException if no animation exists with the given name
     */
    public static void drawAnimationCentered(String name, float centerX, float centerY,
float width, float height) {
        float x = centerX - width / 2f;
        float y = centerY - height / 2f;
        drawAnimation(name, x, y, width, height);
    }

    /**
     * Draws the current frame of the animation centered at the specified position with
size, rotation, and flipping.
     *
     * @param name          the name of the animation
     * @param centerX       the x-coordinate of the center
     * @param centerY       the y-coordinate of the center
     * @param width         the width to draw
     * @param height        the height to draw
     * @param rotationDegrees rotation in degrees (clockwise)
     * @param flipX         whether to flip the frame horizontally
     * @param flipY         whether to flip the frame vertically
     * @throws IllegalArgumentException if no animation exists with the given name
     */
    public static void drawAnimationCentered(String name, float centerX, float centerY,
                                             float width, float height, float rotationDegrees,
                                             boolean flipX, boolean flipY) {
        float x = centerX - width / 2f;
        float y = centerY - height / 2f;
        drawAnimation(name, x, y, width, height, rotationDegrees, flipX, flipY);
    }

    /**
     * Checks if the animation has finished.
     *
     * @param name the animation name
```

```
     * @return true if finished, false otherwise
     * @throws IllegalArgumentException if no animation exists with the given name
     */
    public static boolean isAnimationFinished(String name) {
        SpriteAnimation anim = animations.get(name.toLowerCase());
        if (anim == null) throw new IllegalArgumentException("No animation with name: " +
name);
        return anim.isFinished();
    }

    /**
     * Resets an animation to start from the beginning.
     *
     * @param name the animation name
     */
    public static void resetAnimation(String name) {
        SpriteAnimation anim = animations.get(name.toLowerCase());
        if (anim != null) anim.reset();
    }

    /**
     * Resets an animation to start from the beginning, optionally reversing it.
     *
     * @param name    the animation name
     * @param reverse whether to reverse the animation
     */
    public static void resetAnimation(String name, boolean reverse) {
        SpriteAnimation anim = animations.get(name.toLowerCase());
        if (anim != null) anim.reset(reverse);
    }

    /**
     * Jumps the named animation to a specific frame.
     *
     * @param name      the name of the animation
     * @param frameIndex the 0-based index of the frame
     * @throws IllegalArgumentException if no animation exists with the given name
     * @throws IndexOutOfBoundsException if the frame index is invalid
     */
    public static void jumpAnimationToFrame(String name, int frameIndex) {
        SpriteAnimation anim = animations.get(name.toLowerCase());
```

```
    if (anim == null) throw new IllegalArgumentException("No animation with name: " +
name);
    anim.jumpToFrame(frameIndex);
  }

  /**
   * Returns the index of the current frame of the named animation.
   *
   * @param name the name of the animation
   * @return the 0-based index of the current frame
   * @throws IllegalArgumentException if no animation exists with the given name
   */
  public static int getAnimationCurrentFrameIndex(String name) {
    SpriteAnimation anim = animations.get(name.toLowerCase());
    if (anim == null) throw new IllegalArgumentException("No animation with name: " +
name);
    return anim.getCurrentFrameIndex();
  }

  /**
   * Returns the total number of frames in the named animation.
   *
   * @param name the name of the animation
   * @return the total number of frames
   * @throws IllegalArgumentException if no animation exists with the given name
   */
  public static int getAnimationFrameCount(String name) {
    SpriteAnimation anim = animations.get(name.toLowerCase());
    if (anim == null) throw new IllegalArgumentException("No animation with name: " +
name);
    return anim.getFrameCount();
  }

  //</editor-fold>

  // <editor-fold desc="Mouse cursor">

  /**
   * Hides the system mouse cursor.
   * <p>
   * The cursor will be invisible, but you can still get the mouse coordinates.
```

```
     */
    public static void hideCursor() {
        // Create a 16x16 fully transparent pixmap for the cursor
        Pixmap pixmap = new Pixmap(16, 16, Pixmap.Format.RGBA8888);
        Cursor invisibleCursor = Gdx.graphics.newCursor(pixmap, 0, 0);
        Gdx.graphics.setCursor(invisibleCursor);
        pixmap.dispose();
    }

    /**
     * Shows the default system mouse cursor.
     */
    public static void showCursor() {
        Gdx.graphics.setSystemCursor(Cursor.SystemCursor.Arrow);
    }

    /**
     * Sets a custom cursor image from a file.
     *
     * @param filePath the path to the cursor image (PNG)
     * @param hotspotX the x-coordinate of the active point (tip)
     * @param hotspotY the y-coordinate of the active point (tip)
     */
    public static void setCustomCursor(String filePath, int hotspotX, int hotspotY) {
        Pixmap pixmap = new Pixmap(Gdx.files.internal(filePath));
        Cursor customCursor = Gdx.graphics.newCursor(pixmap, hotspotX, hotspotY);
        Gdx.graphics.setCursor(customCursor);
        pixmap.dispose();
    }

    /**
     * Sets the system cursor to one of the default cursors.
     *
     * @param systemCursor the system cursor to use (e.g., Arrow, Hand, Crosshair,
VerticalResize)
     */
    public static void setSystemCursor(Cursor.SystemCursor systemCursor) {
        Gdx.graphics.setSystemCursor(systemCursor);
    }

    // </editor-fold>
```

```
  // <editor-fold desc="Masking">

  /**
   * Starts a new mask definition. All subsequent shapes or textures
   * will define the mask area.
   *
   * @throws IllegalStateException if sprite or shape rendering is currently active
   */
  public static void startMask() {
    if (maskActive) return;

    if (spriteBatch.isDrawing()) {
      throw new IllegalStateException("Cannot start mask while sprite rendering is
active. Call spriteBatch.end() first.");
    }

    if (shapeRenderer.isDrawing()) {
      throw new IllegalStateException("Cannot start mask while shape rendering is
active. Call shapeRenderer.end() first.");
    }

    maskActive = true;

    Gdx.gl.glEnable(GL20.GL_STENCIL_TEST);
    Gdx.gl.glClear(GL20.GL_STENCIL_BUFFER_BIT);

    // Allow writing to the stencil buffer
    Gdx.gl.glStencilMask(0xFF);
    Gdx.gl.glStencilFunc(GL20.GL_ALWAYS, 1, 0xFF);
    Gdx.gl.glStencilOp(GL20.GL_REPLACE, GL20.GL_REPLACE, GL20.GL_REPLACE);

    // Disable color/depth writing while defining mask
    Gdx.gl.glColorMask(false, false, false, false);
    Gdx.gl.glDepthMask(false);
  }

  /**
   * Ends the mask definition and applies it to subsequent drawing.
   * By default, the mask shows what was drawn in the mask layer.
   */
```

```
    public static void endMask() {
        if (!maskActive) {
            throw new IllegalStateException("Cannot end mask: no active mask. Call
startMask() first.");
        }

        // Re-enable color/depth writing
        Gdx.gl.glColorMask(true, true, true, true);
        Gdx.gl.glDepthMask(true);

        // Configure stencil to show only the masked area
        Gdx.gl.glStencilFunc(GL20.GL_EQUAL, 1, 0xFF);
        Gdx.gl.glStencilMask(0); // stop writing to stencil
        Gdx.gl.glStencilOp(GL20.GL_KEEP, GL20.GL_KEEP, GL20.GL_KEEP);

        maskActive = false;
    }

    /**
     * Inverts the mask effect.
     * Subsequent drawing will render outside the mask instead of inside it.
     */
    public static void invertMask() {
        if (!maskActive) {
            throw new IllegalStateException("Cannot invert mask: no active mask. Call
startMask() first.");
        }

        // Switch stencil function to render outside the mask
        Gdx.gl.glStencilFunc(GL20.GL_NOTEQUAL, 1, 0xFF);
    }

    /**
     * Clears the mask completely.
     * Subsequent drawing will not be affected by any mask.
     */
    public static void clearMask() {
        Gdx.gl.glDisable(GL20.GL_STENCIL_TEST);
    }

    // </editor-fold>
```

```
// <editor-fold desc="Particle effects">

/**
 * Load a particle effect and create a pool for it.
 *
 * @param name     unique name of the effect
 * @param filePath path to the .p file (particle image must be in the same folder)
 */
public static void addParticleEffect(String name, String filePath) {
   if (!particleEffectPools.containsKey(name.toLowerCase())) {
      ParticleEffect effect = new ParticleEffect();
      effect.load(Gdx.files.internal(filePath), Gdx.files.internal(filePath).parent());
      ParticleEffectPool pool = new ParticleEffectPool(effect, 5, 50);
      particleEffectPools.put(name.toLowerCase(), pool);
   }
}

/**
 * Spawn a particle effect at a position.
 *
 * @param name the particle effect name
 * @param x    x-coordinate in the world
 * @param y    y-coordinate in the world
 * @return unique ID of this spawned effect
 */
public static long spawnParticleEffect(String name, float x, float y) {
   ParticleEffectPool pool = particleEffectPools.get(name.toLowerCase());
   if (pool == null) {
      throw new IllegalArgumentException("Particle effect not loaded: " + name);
   }
   ParticleEffectPool.PooledEffect effect = pool.obtain();
   effect.setPosition(x, y);
   long id = nextParticleId++;
   activeParticleEffects.put(id, effect);
   return id;
}

/**
 * Stop a specific active particle effect.
 *
```

```
     * @param id the unique ID returned from spawnParticleEffect
     */
    public static void stopParticleEffect(long id) {
        ParticleEffectPool.PooledEffect effect = activeParticleEffects.remove(id);
        if (effect != null) {
            effect.free();
        }
    }

    /**
     * Update all active particle effects.
     * Uses delta from Gdx.graphics.getDeltaTime().
     */
    public static void updateParticleEffects() {
        float delta = Gdx.graphics.getDeltaTime();
        Array<Long> finishedIds = new Array<>();
        for (Map.Entry<Long, ParticleEffectPool.PooledEffect> entry :
activeParticleEffects.entrySet()) {
            ParticleEffectPool.PooledEffect effect = entry.getValue();
            effect.update(delta);
            if (effect.isComplete()) {
                effect.free();
                finishedIds.add(entry.getKey());
            }
        }
        // Remove finished effects from active map
        for (Long id : finishedIds) {
            activeParticleEffects.remove(id);
        }
    }

    /**
     * Render all active particle effects using GameApp.spriteBatch.
     */
    public static void renderParticleEffects() {
        for (ParticleEffectPool.PooledEffect effect : activeParticleEffects.values()) {
            effect.draw(spriteBatch);
        }
    }

    /**
```

```
    * Draw one specific particle effect instance.
    * In most cases you want to use the method `renderParticleEffects()`, to render all effects at the same time!
    * @param id Id of the particle effect instance
    */
   public static void renderParticleEffect(Long id) {
      ParticleEffectPool.PooledEffect effect = activeParticleEffects.get(id);
      if (effect == null) {
         throw new IllegalArgumentException("Particle effect not found with id: " + id + ". Could it be that it has already finished?");
      }
      effect.draw(spriteBatch);
   }

   /**
    * Checks if the particle effect instance has finished
    * @param id Id of the particle effect instance
    * @return True if it has finished
    */
   public static boolean isParticleEffectFinished(Long id) {
      return activeParticleEffects.containsKey(id);
   }

   /**
    * Clears all active effects and returns them to their pools.
    */
   public static void stopAllActiveParticleEffects() {
      for (ParticleEffectPool.PooledEffect effect : activeParticleEffects.values()) {
         effect.free();
      }
      activeParticleEffects.clear();
   }

   /**
    * Only for advanced users: get the particle pool object from the app directly.
    * With this object you can trigger all particle effects manually.
    * @param name Name of the particle effect
    * @return ParticlePool object
    */
   public static ParticleEffectPool getParticleEffectPool(String name) {
      ParticleEffectPool pool = particleEffectPools.get(name.toLowerCase());
```

```
        if (pool == null) {
            throw new IllegalArgumentException("Particle effect not loaded: " + name);
        }
        return pool;
    }

    /**
     * Get a list with all active particle id's
     * @return List with active particle id's
     */
    public static long[] getActiveParticleEffects() {
        return
activeParticleEffects.keySet().stream().mapToLong(Long::longValue).toArray();
    }

    /**
     * Checks whether a particle effect pool exists for the given name.
     *
     * @param name the particle effect name
     * @return true if the pool exists, false otherwise
     */
    public static boolean hasParticleEffect(String name) {
        return particleEffectPools.containsKey(name.toLowerCase());
    }

    /**
     * Remove the particle effect by name.
     * This will not stop the active particle effect!
     *
     * @param name the name of the particle effect to dispose
     */
    public static void disposeParticleEffect(String name) {
        String key = name.toLowerCase();
        particleEffectPools.remove(key);
    }

    // </editor-fold>

    //<editor-fold desc="Timers">

    /**
```

```
     * Add a timer.
     *
     * @param name unique timer name
     * @param duration duration in seconds
     * @param loops -1 = infinite, 1 = single-shot, >1 = limited loops
     */
    public static void addTimer(String name, float duration, int loops) {
        timers.put(name.toLowerCase(), new Timer(name, duration, loops));
    }

    /**
     * Add a timer with simple looping boolean.
     *
     * @param name timer name
     * @param duration duration in seconds
     * @param looping true = infinite, false = single-shot
     */
    public static void addTimer(String name, float duration, boolean looping) {
        addTimer(name, duration, looping ? -1 : 1);
    }

    /** Returns true if a timer exists. */
    public static boolean hasTimer(String name) {
        return timers.containsKey(name.toLowerCase());
    }

    /** Retrieve timer object. */
    public static Timer getTimer(String name) {
        return timers.get(name.toLowerCase());
    }

    /** Dispose of timer. */
    public static void disposeTimer(String name) {
        timers.remove(name.toLowerCase());
    }

    /** Reset timer to initial state. */
    public static void resetTimer(String name) {
        Timer t = timers.get(name.toLowerCase());
        if (t == null) throw new IllegalArgumentException("No timer with name: " + name);
        t.reset();
```

```
}

/** Pause timer. */
public static void pauseTimer(String name) {
   Timer t = timers.get(name.toLowerCase());
   if (t == null) throw new IllegalArgumentException("No timer with name: " + name);
   t.pause();
}

/** Resume timer. */
public static void resumeTimer(String name) {
   Timer t = timers.get(name.toLowerCase());
   if (t == null) throw new IllegalArgumentException("No timer with name: " + name);
   t.resume();
}

/** Change timer duration. */
public static void setTimerDuration(String name, float newDuration) {
   Timer t = timers.get(name.toLowerCase());
   if (t == null) throw new IllegalArgumentException("No timer with name: " + name);
   t.setDuration(newDuration);
}

/** Get remaining time until next trigger. */
public static float getTimerRemaining(String name) {
   Timer t = timers.get(name.toLowerCase());
   if (t == null) throw new IllegalArgumentException("No timer with name: " + name);
   return t.getRemaining();
}

/** Update all timers using Gdx deltaTime. Call once per frame. */
public static void updateTimers() {
   float delta = Gdx.graphics.getDeltaTime();
   for (Timer t : timers.values()) {
      t.update(delta);
   }
}

/** Update single timer using Gdx deltaTime. */
public static void updateTimer(String name) {
   Timer t = timers.get(name.toLowerCase());
```

```
    if (t == null) throw new IllegalArgumentException("No timer with name: " + name);
    t.updateUsingGdx();
}

/**
 * Query if a timer went off in the last update call.
 * This does NOT update the timer.
 */
public static boolean timerWentOff(String name) {
    Timer t = timers.get(name.toLowerCase());
    if (t == null) throw new IllegalArgumentException("No timer: " + name);
    return t.wentOff();
}

/**
 * Checks whether a timer has finished all its loops.
 * <p>
 * For infinite timers (-1 loops), this always returns false.
 *
 * @param name the timer name
 * @return true if finished, false otherwise
 * @throws IllegalArgumentException if no timer exists with the given name
 */
public static boolean isTimerFinished(String name) {
    Timer t = timers.get(name.toLowerCase());
    if (t == null) throw new IllegalArgumentException("No timer: " + name);
    return t.isFinished();
}

//</editor-fold>

//<editor-fold desc="Debugging">

/**
 * Prints a simple message to the console.
 * @param message the message to print
 */
public static void log(String message) {
    System.out.println(message);
}
```

```
  /**
   * Prints a variable with its name for easier debugging.
   * @param name  the name of the variable
   * @param value the value of the variable
   */
  public static void log(String name, Object value) {
    System.out.println(name + ": " + value);
  }

  /**
   * Prints a warning message in yellow.
   * @param message the warning message
   */
  public static void warn(String message) {
    System.out.println("\u001B[33m[WARNING] " + message + "\u001B[0m"); // Yellow
text
  }

  /**
   * Prints an error message in red.
   * @param message the error message
   */
  public static void error(String message) {
    System.err.println("\u001B[31m[ERROR] " + message + "\u001B[0m"); // Red text
  }

  /**
   * Prints multiple values in a single line.
   * Useful for quick debugging of multiple variables.
   * @param values the values to print (can include labels)
   */
  public static void debug(Object... values) {
    StringBuilder sb = new StringBuilder();
    for (Object v : values) {
      sb.append(v).append(" ");
    }
    System.out.println(sb.toString().trim());
  }

  /**
   * Checks a condition and prints a message if it fails.
```

```
    * Useful for teaching simple assertions.
    * @param condition the condition to test
    * @param message   the message to print if condition is false
    */
   public static void assertCondition(boolean condition, String message) {
      if (!condition) {
         System.out.println("\u001B[31m[ASSERT FAILED] " + message + "\u001B[0m"); // Red text
      }
   }

   //</editor-fold>

   //<editor-fold desc="UI">

   /**
    * Returns the currently active HUD interface.
    *
    * @return HUD instance
    * @throws RuntimeException if the current screen does not implement HUD or HUD is not enabled
    */
   private static HUD getActiveHUD() {
      Screen screen = getCurrentScreen();
      if (!(screen instanceof HUD hud)) {
         throw new RuntimeException("Current screen does not implement HUD.");
      }
      if (!hud.isHUDEnabled()) {
         throw new RuntimeException("HUD not enabled on current screen.");
      }
      return hud;
   }

   /**
    * Adds a skin to the internal skin registry by loading it from a JSON file.
    * The JSON file must reference the associated texture atlas correctly.
    *
    * @param key      unique key to reference the skin later
    * @param jsonPath path to the skin JSON file in the assets folder (e.g., "skins/ui.json")
    * @throws GdxRuntimeException if the skin or atlas cannot be loaded
    */
```

```
public static void addSkin(String key, String jsonPath) {
   Skin skin = new Skin(Gdx.files.internal(jsonPath));
   skins.put(key.toLowerCase(), skin);
}

/**
 * Checks whether a skin exists in the registry.
 *
 * @param key skin key
 * @return true if the skin exists
 */
public static boolean hasSkin(String key) {
   return skins.containsKey(key.toLowerCase());
}

/**
 * Retrieves a skin from the registry by its key.
 *
 * @param key skin key
 * @return Skin object or null if not found
 */
public static Skin getSkin(String key) {
   return skins.get(key.toLowerCase());
}

/**
 * Disposes of a skin in the registry.
 *
 * @param key skin key
 */
public static void disposeSkin(String key) {
   Skin skin = skins.remove(key.toLowerCase());
   if (skin != null) {
      skin.dispose();
   }
}

/**
 * Removes all UI elements from the current HUD stage.
 */
public static void disposeUIElements() {
```

```
    HUD hud = getActiveHUD();
    Stage stage = hud.getUIStage();
    for (Actor actor : uiElements.values()) {
      actor.remove();
    }
    uiElements.clear();
  }

  /**
   * Adds a custom LibGDX {@link Actor} to the currently active HUD.
   * <p>
   * This method can be used to add any UI element that is not yet
   * supported by the convenience methods, such as images, custom widgets,
   * or third-party actors. When the actor triggers any events, you should
   * handle them inside the actor itself or by adding listeners.
   * </p>
   *
   * @param name       unique key for the UI element
   * @param uiComponent the {@link Actor} to add to the HUD stage
   *
   * @throws IllegalStateException if no active HUD is available or HUD is disabled
   * @throws RuntimeException     if an element with the same name already exists
   */
  public static void addCustomUIComponent(String name, Actor uiComponent) {
    HUD hud = getActiveHUD();
    Stage stage = hud.getUIStage();

    if (uiElements.containsKey(name.toLowerCase())) {
      throw new RuntimeException("UI element with name '" + name + "' already
exists.");
    }

    uiElements.put(name.toLowerCase(), uiComponent);
    stage.addActor(uiComponent);
  }

  /**
   * Retrieves the raw LibGDX {@link Actor} for a given UI element key.
   * <p>
   * This works for any UI element added via GameApp's addButton, addSlider,
addCheckBox,
```

```
     * addTextField, addSelectBox, addProgressBar, or addCustomUIComponent
methods.
     * </p>
     *
     * @param name the key of the UI element
     * @return the {@link Actor} associated with the key
     *
     * @throws IllegalArgumentException if no UI element with the given name exists
     */
    public static Actor getUIElement(String name) {
        Actor actor = uiElements.get(name.toLowerCase());
        if (actor == null) {
            throw new IllegalArgumentException("No UI element found with name: " + name);
        }
        return actor;
    }

    /**
     * Checks whether a UI element with the given key exists in the currently active HUD.
     *
     * @param name the key of the UI element
     * @return true if the element exists, false otherwise
     */
    public static boolean hasUIElement(String name) {
        return uiElements.containsKey(name.toLowerCase());
    }

    /**
     * Adds a {@link TextButton} to the currently active HUD.
     * <p>
     * When the button is clicked, {@link HUD#onUIInteraction(String, Object)}
     * will be called with the button's key and a {@code null} value.
     * </p>
     *
     * @param name     unique key for the button
     * @param skinName name of the skin to use
     * @param x        X position in HUD coordinates
     * @param y        Y position in HUD coordinates
     * @param width    button width
     * @param height   button height
     * @param text     text displayed on the button
```

```
     *
     * @throws RuntimeException if the specified skin cannot be found
     * @throws IllegalStateException if no active HUD is available or HUD is disabled
     */
    public static void addButton(String name, String skinName,
                                 float x, float y, float width, float height,
                                 String text) {
        Skin skin = getSkin(skinName);
        if (skin == null) throw new RuntimeException("Skin not found: " + skinName);

        HUD hud = getActiveHUD();
        Stage stage = hud.getUIStage();

        TextButton button = new TextButton(text, skin);
        button.setBounds(x, y, width, height);
        button.addListener(new ChangeListener() {
            @Override
            public void changed(ChangeEvent event, Actor actor) {
                hud.onUIInteraction(name, null);
            }
        });

        uiElements.put(name.toLowerCase(), button);
        stage.addActor(button);
    }

    /**
     * Adds a {@link CheckBox} to the currently active HUD.
     * <p>
     * The checkbox can be used for boolean user input. When its state changes,
     * {@link HUD#onUIInteraction(String, Object)} is called with the checkbox's key
     * and a {@code Boolean} indicating its new state.
     * </p>
     *
     * @param name     unique key for the checkbox
     * @param skinName name of the skin to use
     * @param x        X position in HUD coordinates
     * @param y        Y position in HUD coordinates
     * @param text     label text displayed next to the checkbox
     * @param checked  whether the checkbox is initially checked
     *
```

```
 * @throws RuntimeException if the specified skin cannot be found
 * @throws IllegalStateException if no active HUD is available or HUD is disabled
 */
public static void addCheckBox(String name, String skinName,
                               float x, float y,
                               String text, boolean checked) {
  Skin skin = getSkin(skinName);
  if (skin == null) throw new RuntimeException("Skin not found: " + skinName);

  HUD hud = getActiveHUD();
  Stage stage = hud.getUIStage();

  CheckBox checkBox = new CheckBox(text, skin);
  checkBox.setPosition(x, y);
  checkBox.setChecked(checked);
  checkBox.addListener(new ChangeListener() {
    @Override
    public void changed(ChangeEvent event, Actor actor) {
      hud.onUIInteraction(name, checkBox.isChecked());
    }
  });

  uiElements.put(name.toLowerCase(), checkBox);
  stage.addActor(checkBox);
}

/**
 * Adds a {@link Slider} to the currently active HUD.
 * <p>
 * Sliders allow users to select a value in a numeric range by dragging.
 * When the slider's value changes, {@link HUD#onUIInteraction(String, Object)}
 * is called with the slider's key and current float value.
 * </p>
 *
 * @param name     unique key for the slider
 * @param skinName  name of the skin to use
 * @param x       X position in HUD coordinates
 * @param y       Y position in HUD coordinates
 * @param width    slider width
 * @param height   slider height
 * @param min      minimum slider value
```

```
     * @param max      maximum slider value
     * @param step     increment step size
     * @param vertical  true for vertical, false for horizontal
     *
     * @throws RuntimeException if the specified skin cannot be found
     * @throws IllegalStateException if no active HUD is available or HUD is disabled
     */
    public static void addSlider(String name, String skinName,
                                 float x, float y, float width, float height,
                                 float min, float max, float step, boolean vertical) {
        Skin skin = getSkin(skinName);
        if (skin == null) throw new RuntimeException("Skin not found: " + skinName);

        HUD hud = getActiveHUD();
        Stage stage = hud.getUIStage();

        Slider slider = new Slider(min, max, step, vertical, skin);
        slider.setBounds(x, y, width, height);
        slider.addListener(new ChangeListener() {
            @Override
            public void changed(ChangeEvent event, Actor actor) {
                hud.onUIInteraction(name, slider.getValue());
            }
        });

        uiElements.put(name.toLowerCase(), slider);
        stage.addActor(slider);
    }

    /**
     * Adds a {@link TextField} (input box) to the currently active HUD.
     * <p>
     * Text fields allow the player to type text input. When the value changes,
     * {@link HUD#onUIInteraction(String, Object)} is triggered with the key and the text
string.
     * </p>
     *
     * @param name     unique key for the text field
     * @param skinName name of the skin to use
     * @param x        X position in HUD coordinates
     * @param y        Y position in HUD coordinates
```

```
 * @param width    text field width
 * @param height   text field height
 * @param messageText     initial message that is shown
 *
 * @throws RuntimeException if the specified skin cannot be found
 * @throws IllegalStateException if no active HUD is available or HUD is disabled
 */
public static void addTextField(String name, String skinName,
                                float x, float y, float width, float height,
                                String messageText) {
  Skin skin = getSkin(skinName);
  if (skin == null) throw new RuntimeException("Skin not found: " + skinName);

  HUD hud = getActiveHUD();
  Stage stage = hud.getUIStage();

  TextField textField = new TextField("", skin);
  textField.setBounds(x, y, width, height);
  textField.setMessageText(messageText);
  textField.addListener(new ChangeListener() {
    @Override
    public void changed(ChangeEvent event, Actor actor) {
      hud.onUIInteraction(name, textField.getText());
    }
  });

  uiElements.put(name.toLowerCase(), textField);
  stage.addActor(textField);
}

/**
 * Adds a {@link SelectBox} (drop-down menu) to the currently active HUD.
 * <p>
 * Select boxes let players choose from a list of predefined options.
 * When the selected option changes, {@link HUD#onUIInteraction(String, Object)}
 * is called with the selected value (String).
 * </p>
 *
 * @param name     unique key for the select box
 * @param skinName  name of the skin to use
 * @param x       X position in HUD coordinates
```

```
 * @param y       Y position in HUD coordinates
 * @param width   select box width
 * @param height  select box height
 * @param items   array of selectable items
 *
 * @throws RuntimeException if the specified skin cannot be found
 * @throws IllegalStateException if no active HUD is available or HUD is disabled
 */
public static void addSelectBox(String name, String skinName,
                                float x, float y, float width, float height,
                                String... items) {
  Skin skin = getSkin(skinName);
  if (skin == null) throw new RuntimeException("Skin not found: " + skinName);

  HUD hud = getActiveHUD();
  Stage stage = hud.getUIStage();

  SelectBox<String> selectBox = new SelectBox<>(skin);
  selectBox.setItems(items);
  selectBox.setBounds(x, y, width, height);
  selectBox.addListener(new ChangeListener() {
    @Override
    public void changed(ChangeEvent event, Actor actor) {
      hud.onUIInteraction(name, selectBox.getSelected());
    }
  });

  uiElements.put(name.toLowerCase(), selectBox);
  stage.addActor(selectBox);
}

/**
 * Adds a static {@link Label} to the currently active HUD.
 * <p>
 * Labels display read-only text on the screen, useful for titles or indicators.
 * They do not trigger any {@link HUD#onUIInteraction(String, Object)} events.
 * </p>
 *
 * @param name    unique key for the label
 * @param skinName name of the skin to use
 * @param x       X position in HUD coordinates
```

```
   * @param y       Y position in HUD coordinates
   * @param text    text to display
   *
   * @throws RuntimeException if the specified skin cannot be found
   * @throws IllegalStateException if no active HUD is available or HUD is disabled
   */
  public static void addLabel(String name, String skinName,
                              float x, float y,
                              String text) {
    Skin skin = getSkin(skinName);
    if (skin == null) throw new RuntimeException("Skin not found: " + skinName);

    HUD hud = getActiveHUD();
    Stage stage = hud.getUIStage();

    Label label = new Label(text, skin);
    label.setPosition(x, y);

    uiElements.put(name.toLowerCase(), label);
    stage.addActor(label);
  }

  /**
   * Adds a {@link ProgressBar} to the currently active HUD.
   * <p>
   * The progress bar is a visual indicator of a value between {@code min} and {@code
max}.
   * </p>
   *
   * @param name    unique key for the progress bar
   * @param skinName name of the skin to use
   * @param x       X position in HUD coordinates
   * @param y       Y position in HUD coordinates
   * @param width   progress bar width
   * @param height  progress bar height
   * @param min     minimum value of the progress bar
   * @param max     maximum value of the progress bar
   * @param vertical true if the bar should be vertical, false if horizontal
   * @param value   initial value of the progress bar
   *
   * @throws RuntimeException if the specified skin cannot be found
```

```
     * @throws IllegalStateException if no active HUD is available or HUD is disabled
     */
    public static void addProgressBar(String name, String skinName,
                                      float x, float y, float width, float height,
                                      float min, float max, boolean vertical, float value) {
        Skin skin = getSkin(skinName);
        if (skin == null) throw new RuntimeException("Skin not found: " + skinName);

        HUD hud = getActiveHUD();
        Stage stage = hud.getUIStage();

        ProgressBar progressBar = new ProgressBar(min, max, 0.01f, vertical, skin);
        progressBar.setBounds(x, y, width, height);
        progressBar.setValue(value);

        uiElements.put(name.toLowerCase(), progressBar);
        stage.addActor(progressBar);
    }

    /**
     * Returns the current text of a {@link TextField}.
     *
     * @param name unique element key
     * @return the text currently in the field
     * @throws IllegalArgumentException if no TextField exists with this key
     */
    public static String getTextFieldValue(String name) {
        Actor actor = uiElements.get(name.toLowerCase());
        if (!(actor instanceof TextField)) {
            throw new IllegalArgumentException("No TextField found with key: " + name);
        }
        return ((TextField) actor).getText();
    }

    /**
     * Sets the text of a {@link TextField}.
     *
     * @param name  unique element key
     * @param value text value to set
     * @throws IllegalArgumentException if no TextField exists with this key
     */
```

```
public static void setTextFieldValue(String name, String value) {
   Actor actor = uiElements.get(name.toLowerCase());
   if (!(actor instanceof TextField)) {
      throw new IllegalArgumentException("No TextField found with key: " + name);
   }
   ((TextField) actor).setText(value);
}

/**
 * Returns whether a {@link CheckBox} is currently checked.
 *
 * @param name unique element key
 * @return {@code true} if checked, {@code false} if unchecked
 * @throws IllegalArgumentException if no CheckBox exists with this key
 */
public static boolean getCheckBoxValue(String name) {
   Actor actor = uiElements.get(name.toLowerCase());
   if (!(actor instanceof CheckBox)) {
      throw new IllegalArgumentException("No CheckBox found with key: " + name);
   }
   return ((CheckBox) actor).isChecked();
}

/**
 * Sets the checked state of a {@link CheckBox}.
 *
 * @param name    unique element key
 * @param checked true to check the box, false to uncheck it
 * @throws IllegalArgumentException if no CheckBox exists with this key
 */
public static void setCheckBoxValue(String name, boolean checked) {
   Actor actor = uiElements.get(name.toLowerCase());
   if (!(actor instanceof CheckBox)) {
      throw new IllegalArgumentException("No CheckBox found with key: " + name);
   }
   ((CheckBox) actor).setChecked(checked);
}

/**
 * Returns the current value of a {@link Slider}.
 *
```

```
 * @param name unique element key
 * @return the slider's current float value
 * @throws IllegalArgumentException if no Slider exists with this key
 */
public static float getSliderValue(String name) {
   Actor actor = uiElements.get(name.toLowerCase());
   if (!(actor instanceof Slider)) {
      throw new IllegalArgumentException("No Slider found with key: " + name);
   }
   return ((Slider) actor).getValue();
}

/**
 * Sets the value of a {@link Slider}.
 *
 * @param name  unique element key
 * @param value new value for the slider
 * @throws IllegalArgumentException if no Slider exists with this key
 */
public static void setSliderValue(String name, float value) {
   Actor actor = uiElements.get(name.toLowerCase());
   if (!(actor instanceof Slider)) {
      throw new IllegalArgumentException("No Slider found with key: " + name);
   }
   ((Slider) actor).setValue(value);
}

/**
 * Returns the currently selected item of a {@link SelectBox}.
 *
 * @param name unique element key
 * @return the selected string item
 * @throws IllegalArgumentException if no SelectBox exists with this key
 */
public static String getSelectBoxValue(String name) {
   Actor actor = uiElements.get(name.toLowerCase());
   if (!(actor instanceof SelectBox)) {
      throw new IllegalArgumentException("No SelectBox found with key: " + name);
   }
   return ((SelectBox<?>) actor).getSelected().toString();
}
```

```
  /**
   * Sets the selected item of a {@link SelectBox}.
   *
   * @param name  unique element key
   * @param value the item to select, must be one of the box's available items
   * @throws IllegalArgumentException if no SelectBox exists with this key
   */
  @SuppressWarnings("unchecked")
  public static void setSelectBoxValue(String name, String value) {
    Actor actor = uiElements.get(name.toLowerCase());
    if (!(actor instanceof SelectBox)) {
      throw new IllegalArgumentException("No SelectBox found with key: " + name);
    }
    SelectBox<String> box = (SelectBox<String>) actor;
    if (!box.getItems().contains(value, false)) {
      throw new IllegalArgumentException("Value '" + value + "' not in SelectBox items
for key: " + name);
    }
    box.setSelected(value);
  }

  /**
   * Returns the text of a {@link Label}.
   *
   * @param name unique element key
   * @return the text displayed by the label
   * @throws IllegalArgumentException if no Label exists with this key
   */
  public static String getLabelValue(String name) {
    Actor actor = uiElements.get(name.toLowerCase());
    if (!(actor instanceof Label)) {
      throw new IllegalArgumentException("No Label found with key: " + name);
    }
    return ((Label) actor).getText().toString();
  }

  /**
   * Sets the text of a {@link Label}.
   *
   * @param name  unique element key
```

```
     * @param value new text to display
     * @throws IllegalArgumentException if no Label exists with this key
     */
    public static void setLabelValue(String name, String value) {
        Actor actor = uiElements.get(name.toLowerCase());
        if (!(actor instanceof Label)) {
            throw new IllegalArgumentException("No Label found with key: " + name);
        }
        ((Label) actor).setText(value);
    }

    /**
     * Retrieves the current value of a {@link ProgressBar} on the active HUD.
     *
     * @param name unique key of the progress bar
     * @return current value of the progress bar
     * @throws IllegalArgumentException if no progress bar with the specified name exists
     */
    public static float getProgressBarValue(String name) {
        Actor actor = uiElements.get(name.toLowerCase());
        if (actor instanceof ProgressBar pb) {
            return pb.getValue();
        } else {
            throw new IllegalArgumentException("No progress bar found with name: " +
name);
        }
    }

    /**
     * Sets the value of a {@link ProgressBar} on the active HUD.
     *
     * @param name  unique key of the progress bar
     * @param value new value to set; will be clamped to the progress bar's min and max
     * @throws IllegalArgumentException if no progress bar with the specified name exists
     */
    public static void setProgressBarValue(String name, float value) {
        Actor actor = uiElements.get(name.toLowerCase());
        if (actor instanceof ProgressBar pb) {
            pb.setValue(value);
        } else {
            throw new IllegalArgumentException("No progress bar found with name: " +
```

```
name);
      }
   }

   //</editor-fold>
}
```